

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf\_e5140909-890\agent-a96fa2e6abb53a4ca.jsonl`
- SHA-256: `e28035c0efe137b259410beb20a4a959ffe6fab963288157337ad4f5575a1386`
- Source modified: `2026-06-18T07:57:31+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:54:35.125Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
You are a senior engineer scoping a SAFE refactoring effort. The owner wants critical tech debt cleaned up in files that "grew a lot", with ZERO behavior change and NO quality regression. Propose only refactors that are: (a) behavior-PRESERVING (pure moves/extractions/dedup/dead-code removal ? NOT redesigns, NOT logic changes), (b) backed by EXISTING tests that would catch a regression (assess this honestly per proposal ? if coverage is weak/none, say so and rate risk higher), (c) cleanly separable into ONE reviewable PR. For each: the exact seam (what moves where), the behavior-preservation argument, honest test_coverage, risk, est diff size, value. PREFER high-value + low-risk + well-tested. It is FINE to say a giant function should NOT be refactored yet because coverage is too thin ? flag that as a 'needs tests first' note in debt.

Signals (LOC / longest-functions): gemini.process_video=434L/71br, yolo_hybrid=309L, trajectory_hybrid=231L, motion=204L, compiler.compile_highlights=202L, golden_fixtures=1113L (freeze_real_fixture=186L), main.py=871L (main=168L, _execute_processing=112L), database.py=738L (_init_db=187L), wasb_infer=699L (run=185L), ablation=620L, experiment=579L, tracknet_runner=506L.

CLUSTER:

backend/pipeline/detectors/{wasb_infer.py(699),tracknet_runner.py(506),person_detector.py} + stitcher/compiler.py(202L compile_highlights). Extract-method on the long run/window functions; dedup; dead code. Assess GPU-path test coverage (much is owner-side) and rate risk accordingly.

2. assistant (2026-06-18T07:54:37.988Z)

I'll scope safe refactors in the named cluster. Let me start by pulling the latest and reading the actual files.

3. user (2026-06-18T07:54:40.156Z)

a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)

```
---
__init__.py
__pycache__
audio_features.py
base.py
fusion_features.py
person_detector.py
rally_gate.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py
```

---STITCHER---
compiler.py

4. assistant (2026-06-18T07:54:42.538Z)

Let me read the main cluster files. I'll start with the largest ones and their tests.

5. user (2026-06-18T07:54:43.860Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40
41     """
42     import os
43     import os.path as osp
44     import csv
45     import glob
46     import json
47     import argparse
48     import logging
49     from collections import defaultdict
50     from contextlib import contextmanager
51
52     logger = logging.getLogger(__name__)
53
54     CACHE_VERSION = 1
55     _DET_FILE = "det_raw.jsonl"
56     _MANIFEST_FILE = "manifest.json"
```

```

55
56 def _build_cfg(weights: str, sport: str):
57     """Compose the WASB eval config via Hydra, overriding model/weights/device."""
58     from hydra import compose, initialize
59     with initialize(config_path="configs", version_base=None):
60         cfg = compose(config_name="eval", overrides=[
61             f"dataset={sport}",
62             "model=wasb",
63             f"detector.model_path={weights}",
64             "runner.device=cuda",
65             "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
66         ])
67     return cfg
68
69
70 def _frame_id(path: str) -> int:
71     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72     stem = osp.splitext(osp.basename(path))[0]
73     digits = "".join(ch for ch in stem if ch.isdigit())
74     return int(digits) if digits else -1
75
76
77 # ----- #
78 # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79 # ----- #
80 def _cache_paths(cache_dir: str):
81     return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84 def default_cache_dir(out_csv: str) -> str:
85     """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86     d = osp.dirname(osp.abspath(out_csv))
87     stem = osp.splitext(osp.basename(out_csv))[0]
88     return osp.join(d, stem + "_wasbcache")
89
90
91 def _atomic_write_text(path: str, text: str) -> None:
92     """Write then rename so a crash mid-write can't leave a half-written file."""
93     tmp = path + ".tmp"
94     with open(tmp, "w") as f:
95         f.write(text)
96         f.flush()
97         os.fsync(f.fileno())
98     os.replace(tmp, path)
99
100
101 def _det_plain(det) -> list:
102     """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
103     xy = det["xy"]
104     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106
107 def _dets_to_tracker(plain_list):
108     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
109     the tracker does vector math / np.linalg.norm on it)."""
110     import numpy as np
111     return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112             for p in plain_list]
113
114
115 def write_manifest(cache_dir: str, manifest: dict) -> None:

```

```

116     _, mpath = _cache_paths(cache_dir)
117     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120 def read_manifest(cache_dir: str):
121     _, mpath = _cache_paths(cache_dir)
122     if not osp.exists(mpath):
123         return None
124     try:
125         with open(mpath) as f:
126             return json.load(f)
127     except (json.JSONDecodeError, OSError):
128         return None
129
130
131 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                         frames_in: int, frames_total: int) -> bool:
133     """A cache is reusable only if the run that produced it matches this run's
134 inputs."""
135     if not manifest or manifest.get("version") != CACHE_VERSION:
136         return False
137     return (
138         manifest.get("frames_dir") == osp.abspath(frames_dir)
139         and manifest.get("weights") == weights
140         and manifest.get("sport") == sport
141         and manifest.get("frames_in") == frames_in
142         and manifest.get("frames_total") == frames_total
143     )
144
145 def reset_cache(cache_dir: str) -> None:
146     """Drop any existing cache so the next run starts clean."""
147     det_jsonl, mpath = _cache_paths(cache_dir)
148     for p in (det_jsonl, mpath):
149         if osp.exists(p):
150             os.remove(p)
151
152
153 def load_and_clean_cache(cache_dir: str):
154     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156     prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159     [x, y, score, scale] candidates (accumulated across every window the frame is in,
160     in window order ? identical to a non-resumed run).
161     """
162     det_jsonl, _ = _cache_paths(cache_dir)
163     det_by_fid = defaultdict(list)
164     valid: list = []
165     if osp.exists(det_jsonl):
166         with open(det_jsonl, "r") as f:
167             for line in f:
168                 s = line.strip()
169                 if not s:
170                     continue
171                 try:
172                     obj = json.loads(s)
173                 except json.JSONDecodeError:
174                     break # truncated trailing line (crash mid-
flush)
175                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                     break
177                 valid.append(s)
178                 for fid, plain in obj["f"]:

```

```

179         det_by_fid[int(fid)].extend([list(p) for p in plain])
180         # Guarantee a clean append boundary by rewriting only the good prefix.
181         _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182     return len(valid), det_by_fid
183
184
185 def _append_windows(fh, objs) -> None:
186     """Append window records as JSONL and durably flush (bounded loss on crash)."""
187     for o in objs:
188         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189     fh.flush()
190     os.fsync(fh.fileno())
191
192
193 def _atomic_write_trajectory(out_csv: str, rows) -> None:
194     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)
200         f.flush()
201         os.fsync(f.fileno())
202     os.replace(tmp, out_csv)
203
204
205 # ----- #
206 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207 # ----- #
208 _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211 def synthetic_frame_path(index: int) -> str:
212     """Stable, index-encoding frame name used by the streaming path.
213
214     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
215     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216     frames came from disk or from the in-memory stream. The streaming image loader
217     inverts this name back to an index to fetch the decoded frame.
218     """
219     return _STREAM_FRAME_FMT.format(int(index))
220
221
222 def build_windows(frames: list, frames_in: int):
223     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224     + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226     Returns a list of windows, each a list of ``frames_in`` consecutive paths
227     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230     """
231     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234 class SequentialFrameStore:
235     """In-memory sliding buffer over a forward-only frame reader, sized for the
236     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the

```

```

244     earliest index any not-yet-finished window can still ask for) and decode each source
245     frame exactly once via the forward-only ``reader(index)``.
246     """
247
248     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249         self._reader = reader
250         self._total = int(total)
251         self._window = max(1, int(window))
252         self._buf: dict = {}
253         # On a resumed run the detector's first needed frame is `start_index`; begin
254         # pulling there so the reader can skip (seek past) the already-cached prefix
255         # instead of decoding 0..start_index-1 just to throw them away.
256         self._next = int(start_index)      # next index not yet pulled from the reader
257         self._max_seen = int(start_index) - 1
258         self._floor = int(start_index)     # lowest index we still keep (never evict
below)
259
260     def get(self, index: int):
261         index = int(index)
262         if index < self._floor:
263             raise KeyError(
264                 f"frame {index} already evicted (below floor {self._floor}); the access
"
265                 f"pattern must stay within a window of the highest frame seen")
266         if index >= self._total:
267             raise KeyError(f"frame {index} out of range (total {self._total})")
268         # Pull forward from the reader until the requested index is buffered.
269         while self._next <= index:
270             self._buf[self._next] = self._reader(self._next)
271             self._next += 1
272         if index > self._max_seen:
273             self._max_seen = index
274         # Evict frames older than the earliest a still-running window could re-request:
275         # once we've seen index `m`, no future window starts before `m - (window-1)`.
276         new_floor = max(self._floor, self._max_seen - (self._window - 1))
277         for k in range(self._floor, new_floor):
278             self._buf.pop(k, None)
279         self._floor = new_floor
280         return self._buf[index]
281
282
283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291
292     def _bgr_to_pil_rgb(frame_bgr):
293         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
297         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
298         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
299         frame indistinguishable from the on-disk one to everything downstream.
300         """
301         import cv2
302         from PIL import Image

```

```

303         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
304
305
306     def _make_streamed_read_image(store, real_read_image):
307         """Build the ``read_image`` replacement used during a streaming detector pass.
308
309         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
310 image,
311 matching ``read_image``'s real return type); any path that doesn't parse to an index
312 falls
313 through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
314 testable
315 without the WSL-only ``dataloaders`` package.
316 """
317     def _read_image(path, *args, **kwargs):
318         idx = _index_from_synthetic_path(path)
319         if idx < 0:
320             return real_read_image(path, *args, **kwargs)
321         return _bgr_to_pil_rgb(store.get(idx))
322     return _read_image
323
324 @contextmanager
325 def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
326 int = 0):
327     """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
328 decoded
329 frames during the detector pass, byte-for-byte as it would over real PNGs.
330
331 WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
332 ``cv2.imread``. ``read_image`` (``utils.utils``) does
333 ``Image.open(path).convert('RGB')``,
334 returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
335 feed it a
336 PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
337 byte-identical to the disk round-trip, and the shim never touches the filesystem so
338 the
339 ``osp.exists`` guard is sidestepped for synthetic stream paths.
340
341 We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
342 actually
343 calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
344 in that
345 module's namespace; patching ``utils.read_image`` would NOT rebind the already-
346 imported
347 reference).
348
349 ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
350 wrapped
351 in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
352 on a
353 resume we start at ``start_index`` so the decoder seeks past already-cached windows.
354 The
355 original reader is restored on exit.
356 """
357     from dataloaders import dataset_loader as _dl
358     store = SequentialFrameStore(frame_loader, total, window=window,
359 start_index=start_index)
360     real_read_image = _dl.read_image
361     # Rebind read_image in the consuming module for the duration of the detector pass;
362     # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
363 B010-clean.)
364     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
365 ignore[assignment]
366     try:

```

```

351         yield store
352     finally:
353         _dl.read_image = real_read_image # type: ignore[assignment]
354
355
356     # ----- #
357     # Inference
358     # ----- #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
360             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361             log_every_batches: int = 50, cache_dir: "str | None" = None,
362             flush_every: int = 1000, fresh: bool = False,
363             frame_loader=None, source_key: "str | None" = None) -> str:
364         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366         Two equivalent ways to supply pixels (output is bit-identical between them):
367
368         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
370           ``cv2.imread``.
371         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
372           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
373           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
374           reader, which returns a PIL RGB image) over the DataLoader pass so every other
375           line of
376           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
377           identical
378           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
379           ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
380           ``num_workers=0``
381           (the in-process shim + buffer can't cross worker processes).
382
383         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
384         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
385         after reboot still matches.
386         """
387         import time
388         import torch
389         from torch.utils.data import DataLoader
390         from detectors import build_detector
391         from trackers import build_tracker
392         from dataloaders import build_img_transforms, build_seq_transforms
393         from dataloaders.dataset_loader import ImageDataset
394         from utils import Center
395
396         streaming = frame_loader is not None
397
398         cfg = _build_cfg(weights, sport)
399         frames_in = int(cfg["model"]["frames_in"])
400         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
401         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
402
403         if streaming:
404             # ``frames`` is already the ordered list of (synthetic) frame paths.
405             if not isinstance(frames, (list, tuple)):
406                 raise SystemExit("streaming mode requires ``frames`` to be a list of frame
407                 paths")
408             frames = list(frames)
409             frames_dir = source_key or "stream"
410         else:
411             frames_dir = frames
412             frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
413             glob.glob(osp.join(frames_dir, "*.jpg"))
414             if limit:
415                 frames = frames[:limit]

```

```

410     if len(frames) < frames_in:
411         where = frames_dir if not streaming else "video stream"
412         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
413
414     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
415     samples = []
416     for window in build_windows(frames, frames_in):
417         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
418         samples.append({"frames": window, "annos": annos})
419     windows_total = len(samples)
420
421     # ---- cache setup / resume decision ----- #
422     if cache_dir is None:
423         cache_dir = default_cache_dir(out_csv)
424         os.makedirs(cache_dir, exist_ok=True)
425         det_jsonl, _ = _cache_paths(cache_dir)
426         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427         manifest = None if fresh else read_manifest(cache_dir)
428         resume = manifest_compatible(
429             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430             frames_in=frames_in, frames_total=len(frames),
431         )
432         if resume:
433             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435         else:
436             if manifest is not None:
437                 logger.info("cache incompatible with this run -> starting fresh")
438                 reset_cache(cache_dir)
439                 windows_done, det_by_fid = 0, defaultdict(list)
440
441     base_manifest = {
442         "version": CACHE_VERSION,
443         "frames_dir": cache_key,
444         "weights": weights,
445         "sport": sport,
446         "frames_in": frames_in,
447         "frames_total": len(frames),
448         "windows_total": windows_total,
449         "windows_done": windows_done,
450     }
451     write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462             transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463         # Streaming forces single-process loading: the in-memory frame store + the
464         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465         loader_workers = 0 if streaming else num_workers
466         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467         detector = build_detector(cfg)
468

```

```

469         from contextlib import ExitStack
470
471         t0 = time.time()
472         done = windows_done
473         win_idx = windows_done
474         log_every = max(1, log_every_batches)
475         flush_n = max(1, flush_every)
476         pending = []
477         logger.info(f"detecting on {len(remaining)} remaining windows "
478                   f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479                   f"source={'video-stream' if streaming else 'frames-dir'})...")
480         det_f = open(det_jsonl, "a")
481         try:
482             with ExitStack() as stack:
483                 stack.enter_context(torch.no_grad())
484                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485                 # in-memory decoded frame for that synthetic path; every other line of
486                 # `__getitem__` runs unchanged so the tensor matches the PNG round-trip
exactly.
487                 # The detector's first needed frame is `windows_done` (window i starts
at
488                 # frame i), so prime the store there for a resume.
489                 if streaming:
490                     stack.enter_context(
491                         _streaming_read_image_patch(frame_loader, len(frames),
492                                                     window=frames_in,
start_index=windows_done))
493                 for bi, batch in enumerate(loader):
494                     imgs, _hms, trans = batch[0], batch[1], batch[2]
495                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496                     batch_results, _ = detector.run_tensor(imgs, trans)
497                     nb = imgs.shape[0]
498                     for ib in range(nb):
499                         frames_payload = []
500                         for ie in sorted(batch_results[ib].keys()):
501                             p = img_paths[ie][ib]
502                             fid = _frame_id(p)
503                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                             frames_payload.append([fid, plain])
505                             det_by_fid[fid].extend(plain)
506                             pending.append({"w": win_idx, "f": frames_payload})
507                             win_idx += 1
508                     done += nb
509                     if len(pending) >= flush_n or done >= windows_total:
510                         _append_windows(det_f, pending)
511                         pending = []
512                         base_manifest["windows_done"] = done
513                         write_manifest(cache_dir, base_manifest)
514                     if (bi + 1) % log_every == 0 or done >= windows_total:
515                         el = time.time() - t0
516                         new_done = done - windows_done
517                         rate = new_done / el if el > 0 else 0.0
518                         eta = (windows_total - done) / rate if rate > 0 else 0.0
519                         logger.info(f"{done}/{windows_total} "
520                                   f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                                   f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522                     if pending:
523                         _append_windows(det_f, pending)
524                         base_manifest["windows_done"] = done
525                         write_manifest(cache_dir, base_manifest)
526         finally:

```

```

527         det_f.close()
528         logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529     else:
530         logger.info("detector cache already complete -> tracker-only re-run")
531
532     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533     tracker = build_tracker(cfg)
534     tracker.refresh()
535     rows = []
536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:
548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578 def _probe_frame_count(video_path: str) -> int:
579     """Container-reported frame count via cv2 (0 if unknown). Used to size the
580     stream."""
581     import cv2
582     cap = cv2.VideoCapture(video_path)
583     if not cap.isOpened():
584         raise SystemExit(f"cannot open video: {video_path}")
585     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586     cap.release()
587     return n
588
589 def make_video_frame_loader(video_path: str):
590     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

```

```

591
592 ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
593 long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594 detector's sliding-window order); it reads sequentially and only uses a frame-peek
595 to
596 skip forward when a *resume* starts past the current position. Each ``cap.read()``
597 returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
598 would
599 yield (PNG is lossless), so the detector sees identical pixels to the disk path.
600 Returns the count hint from the container so the caller can build the frame list;
601 ``release`` closes the capture.
602 """
603 import cv2
604 cap = cv2.VideoCapture(video_path)
605 if not cap.isOpened():
606     raise SystemExit(f"cannot open video: {video_path}")
607 count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
608 state = {"pos": 0} # next index cap.read() will return
609
610 def frame_loader(index: int):
611     index = int(index)
612     if index < state["pos"]:
613         raise RuntimeError(
614             f"streaming decode is forward-only; got index {index} after
615 {state['pos']}")
616     if index > state["pos"]:
617         # Forward skip only happens when resuming past cached windows. Seek once.
618         cap.set(cv2.CAP_PROP_POS_FRAMES, index)
619         state["pos"] = index
620     ok, frame = cap.read()
621     if not ok:
622         raise RuntimeError(f"failed to decode frame {index} from {video_path}")
623     state["pos"] += 1
624     return frame
625
626 def release():
627     cap.release()
628
629 return frame_loader, count_hint, release
630
631 def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
632 "badminton",
633                          limit: int = 0, batch_size: int = 8,
634                          log_every_batches: int = 50, cache_dir: "str | None" = None,
635                          flush_every: int = 1000, fresh: bool = False) -> str:
636     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
637     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
638     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
639     ``run``'s docstring for why.
640     """
641     frame_loader, count_hint, release = make_video_frame_loader(video_path)
642     try:
643         if count_hint <= 0:
644             raise SystemExit(
645                 f"could not determine frame count for {video_path}; cannot stream "
646                 f"(fall back to frame extraction with --frames_dir)")
647         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
648         source_key = "video:" + osp.abspath(video_path)
649         return run(frame_list, weights, out_csv, sport, limit,
650                   batch_size=batch_size, num_workers=0,
651                   log_every_batches=log_every_batches, cache_dir=cache_dir,
652                   flush_every=flush_every, fresh=fresh,
653                   frame_loader=frame_loader, source_key=source_key)

```

```

652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662             help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                 "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668         ap.add_argument("--batch-size", type=int, default=8)
669         ap.add_argument("--num-workers", type=int, default=4)
670         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671         ap.add_argument("--cache-dir", default=None,
672             help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673         ap.add_argument("--flush-every", type=int, default=1000,
674             help="fsync the detector cache every N windows (bounds reboot
loss)")
675         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676         args = ap.parse_args()
677
678         if args.stream_video:
679             if not args.video:
680                 raise SystemExit("--stream-video requires --video")
681             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                 batch_size=args.batch_size,
683                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                 flush_every=args.flush_every, fresh=args.fresh)
685             return
686
687         frames_dir = args.frames_dir
688         if args.video:
689             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690         if not frames_dir:
691             raise SystemExit("provide --frames_dir or --video")
692         run(frames_dir, args.weights, args.out, args.sport, args.limit,
693             batch_size=args.batch_size, num_workers=args.num_workers,
694             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695             flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

6. assistant (2026-06-18T07:54:46.226Z)

Now let me read the tracknet_runner, person_detector, and compiler, plus find their tests.

7. user (2026-06-18T07:54:47.419Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                        # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                       # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                       # warn if WSL free space below this
(0 = off)
44
45      @classmethod
46      def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47          tn = (idx_cfg or {}).get("tracknet", {}) or {}
48          return cls(
49              distro=tn.get("wsl_distro", "Ubuntu"),
50              repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51              conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52              conda_env=tn.get("conda_env", "TrackNetV4"),
53              weights_path=tn.get("weights_path", ""),
54              queue_length=int(tn.get("queue_length", 5)),
55              stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
```

```

56         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57         timeout_sec=int(tn.get("timeout_sec", 1800)),
58         keep_frames=bool(tn.get("keep_frames", False)),
59         min_free_gb=float(tn.get("min_free_gb", 0.0)),
60     )
61
62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "`wsl` not found ? is this running on Windows with WSL2 installed?"
112             except subprocess.TimeoutExpired:
113                 return False, "WSL healthcheck timed out."
114             out = (res.stdout or "").strip()
115             if res.returncode != 0:
116                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
117             return True, out
118
119 # ----- inference

```

```

119
120 def run_predict(self, video_win_path: str, output_win_dir: str,
121                 video_id: Optional[str] = None) -> Optional[str]:
122     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
123
124     ``output_win_dir`` is a Windows directory (under the repo's output/) that
125     WSL writes into via its /mnt view, so the Windows process can read the CSV
126     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
127     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
128     a filename cannot collide on the output CSV.
129     """
130     if not self.cfg.weights_path:
131         logger.error(
132             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133             "in config.json to the WSL path of your trained TrackNetV4 weights."
134         )
135         return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass Windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}__{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "

```

```

182         f"python predict.py "
183         f"--video_path {wsl_tilde_quote(wsl_video)} "
184         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185         f"--output_dir {wsl_tilde_quote(wsl_out)} "
186         f"--queue_length {self.cfg.queue_length}"
187     )
188     logger.info("Running TrackNet inference on %s ...", base)
189     try:
190         res = self._wsl_bash(cmd)
191     except subprocess.TimeoutExpired:
192         logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193         return None
194
195     if res.returncode != 0:
196         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197         return None
198
199     if not os.path.exists(expected_csv_win):
200         logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201         return None
202     logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205     # regenerable intermediate). On earlier failure we return above without
cleaning.
206     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207         logger.info("Cache hygiene: removing staged video for %s "
208                     "(set indexing.tracknet.keep_frames=true to retain).", key)
209         self._wsl_rm_rf([wsl_video])
210     return expected_csv_win
211
212 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215     Returns the WSL path to the staged copy, or None on failure.
216     """
217     src_mnt = to_wsl_mnt_path(video_win_path)
218     reason = self.wsl_source_unreadable_reason(src_mnt)
219     if reason:
220         logger.error("Cannot stage video into WSL: %s", reason)
221         return None
222     dest = f"{self.cfg.wsl_stage_dir}/{base}"
223     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224     try:
225         res = self._wsl_bash(cmd)
226     except subprocess.TimeoutExpired:
227         logger.error("Staging video into WSL timed out.")
228         return None
229     if res.returncode != 0 or "OK" not in (res.stdout or ""):
230         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231         return None
232     return dest
233
234 # ----- CSV -> windows
235
236 @staticmethod
237 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239     points: List[TrajectoryPoint] = []
240     with open(csv_win_path, newline="") as f:

```

```

241         reader = csv.DictReader(f)
242         for row in reader:
243             try:
244                 points.append(TrajectoryPoint(
245                     frame=int(row["Frame"]),
246                     visible=int(row["Visibility"]) == 1,
247                     x=float(row["X"]),
248                     y=float(row["Y"]),
249                 ))
250             except (KeyError, ValueError):
251                 continue
252         points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,
259         frame_width: float,
260         chunk_start: float = 0.0,
261         velocity_thresh: float = 0.02,
262         merge_gap: float = 2.0,
263         min_window_duration: float = 2.0,
264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267         window_hard_cap: bool = False,
268         window_inactivity_end: float = 0.0,
269         inactivity_min_density: float = 0.34,
270         inactivity_rally_thresh: float = 0.08,
271         window_blob_fallback: bool = False,
272         window_blob_factor: float = 4.0,
273     ) -> List[Dict[str, Any]]:
274         """Convert a shuttle trajectory into candidate rally windows.
275
276         A frame is "active" when the shuttle is visible AND its inter-frame
277         displacement (normalised by frame width, per second) exceeds
278         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279         ``merge_gap`` seconds of each other are grouped into one window; short
280         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281         shape feeding the AI-handoff phase is identical.
282
283         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
284         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
285         progress" and a single window swallows several rallies + the dead time between them. When
286         set, any window longer than this is recursively SPLIT at its largest internal inactivity
287         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
288         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
289         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
291
292         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
293         an over-long window has NO internal inactivity gap to split on (a continuously-
294         active trajectory ? the shuttle never stops moving, e.g. the real-footage
295         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively

```

```

296         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
297         (never merges), so recall cannot drop; gated default-OFF until measured.
298
299         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
300         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
301         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
302         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` * x
303         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
304         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
305         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
306         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
307         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
308         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
309         """
310         if fps <= 0:
311             fps = 30.0
312         if frame_width <= 0:
313             frame_width = 1280.0
314
315         # Compute per-frame normalised velocity from the last *visible* point.
316         active = [] # (frame_idx, velocity)
317         prev: Optional[TrajectoryPoint] = None
318         for p in points:
319             if not p.visible:
320                 prev = None # break the trajectory; absent shuttle = not in flight
321                 continue
322             if prev is not None:
323                 dframes = p.frame - prev.frame
324                 if dframes > 0:
325                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                     velocity = (dist / frame_width) * (fps / dframes)
327                     if velocity > velocity_thresh:
328                         active.append((p.frame, velocity))
329             prev = p
330
331         if not active:
332             return []
333
334         max_gap_frames = int(merge_gap * fps)
335
336         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337             vels = [v for _, v in group]
338             return {
339                 "start": group[0][0] / fps + chunk_start,
340                 "end": group[-1][0] / fps + chunk_start,
341                 "active_frame_count": len(group),
342                 "peak_velocity": max(vels),
343                 "mean_velocity": sum(vels) / len(vels),
344                 "track_id_count": 1, # single shuttle; kept for window-shape parity
345                 "chunk_index": chunk_index,
346             }
347
348         groups: List[List[Tuple[int, float]]] = []
349         group = [active[0]]

```

```

350         for af in active[1:]:
351             if af[0] - group[-1][0] <= max_gap_frames:
352                 group.append(af)
353             else:
354                 groups.append(group)
355                 group = [af]
356         groups.append(group)
357
358         if max_window_duration > 0:
359             groups = TrackNetRunner._split_overlong_groups(
360                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
361                 hard_cap=window_hard_cap)
362
363         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368         if window_inactivity_end > 0:
369             tw = int(window_inactivity_end * fps)
370             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                                                         inactivity_rally_thresh) for g
in groups]
372
373         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377         forced_flags = [False] * len(groups)
378         if window_blob_fallback and max_window_duration > 0:
379             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380                 groups, int(max_window_duration * fps), window_blob_factor)
381
382         windows = []
383         for g, forced in zip(groups, forced_flags):
384             w = _finalize(g)
385             if forced:
386                 w["forced_cut"] = True
387             windows.append(w)
388         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
393         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
394         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
395         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
396         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
397         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
398         LAST such frame ? everything after is sustained slow drift (rally over). Only

```

```

cuts (recall
399         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
400         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
401         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
402         if trail_frames <= 0 or len(group) < 2:
403             return group
404         frames = [f for f, _ in group]
405         fast = [1 if v > rally_thresh else 0 for _, v in group]
406         lo = 0
407         fast_sum = 0
408         last_dense = -1
409         for i in range(len(frames)):
410             fast_sum += fast[i]
411             while frames[lo] <= frames[i] - trail_frames:
412                 fast_sum -= fast[lo]
413                 lo += 1
414             win_count = i - lo + 1
415             if win_count > 0 and fast_sum / win_count >= min_density:
416                 last_dense = i
417         return group[: last_dense + 1] if last_dense >= 0 else group
418
419     @staticmethod
420     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                               min_split_frames: int,
422                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
423         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
428         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
429         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430         if max_win_frames <= 0:
431             return groups
432         out: List[List[Tuple[int, float]]] = []
433         work = list(groups)
434         while work:
435             g = work.pop()
436             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                 out.append(g)
438                 continue
439             best_i, best_gap = -1, -1
440             for i in range(1, len(g)):
441                 gap = g[i][0] - g[i - 1][0]
442                 if gap > best_gap:
443                     best_gap, best_i = gap, i
444             if best_i < 0 or best_gap < min_split_frames:
445                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447                 if hard_cap:
448                     mid = (g[0][0] + g[-1][0]) / 2.0
449                     split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450                     work.append(g[:split_i])

```

```

451         work.append(g[split_i:])
452     else:
453         out.append(g)
454         continue
455         work.append(g[:best_i])
456         work.append(g[best_i:])
457     out.sort(key=lambda grp: grp[0][0])
458     return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
466         ``blob_factor`` x
467         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
468         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
469         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
470         (~1?2x the cap) is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
471         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
472         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
473         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
474         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
475         that piece came from a forced chop. Iterative worklist to bound stack depth."""
476         if max_win_frames <= 0:
477             return groups, [False] * len(groups)
478         blob_frames = max_win_frames * max(1.0, blob_factor)
479         out: List[List[Tuple[int, float]]] = []
480         flags: List[bool] = []
481         forced = 0
482         for g in groups:
483             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
484                 out.append(g)
485                 flags.append(False)
486                 continue
487             pieces: List[List[Tuple[int, float]]] = []
488             work = [g]
489             while work:
490                 h = work.pop()
491                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
492                     pieces.append(h)
493                     continue
494                     mid = (h[0][0] + h[-1][0]) / 2.0
495                     split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
496                     work.append(h[:split_i])
497                     work.append(h[split_i:])
498                     pieces.sort(key=lambda grp: grp[0][0])
499                     out.extend(pieces)
500                     flags.extend([True] * len(pieces))
501                     forced += len(pieces) - 1
502             if forced:
503                 logger.warning(
504                     "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
505                     "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /

```

```

two-sided "
505             "motion), not a bigger cap", forced)
506         return out, flags
507

```

8. assistant (2026-06-18T07:54:49.407Z)

Now the compiler and person_detector, plus locate the test files.

9. user (2026-06-18T07:54:50.472Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)
26
27        @staticmethod
28        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29            if watermark is None:
30                return None
31            if hasattr(watermark, "model_dump"):
32                return watermark.model_dump()
33            if isinstance(watermark, dict):
34                return dict(watermark)
35            return None
36
37        @staticmethod
38        def _ff_quote(path: str) -> str:
39            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41            slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44            option parser treats ':' as the option separator, so a Windows drive-letter
45            path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46            the encode fails with "Invalid argument". This silently disabled the
47            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48            return path.replace("\\", "/").replace("'", r'\').replace(":", r"\:")
49

```

```

50     @staticmethod
51     def _parse_time(val: Union[int, float, str]) -> float:
52         """Normalizes a timestamp value to float seconds.
53         Handles: float/int (pass-through), plain numeric strings ("12.4"),
54         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55         This mirrors the parse_time logic used in gemini.py to ensure
56         the stitcher can always consume whatever format the indexer produced."""
57         if isinstance(val, (int, float)):
58             return float(val)
59         if isinstance(val, str):
60             val = val.strip()
61             if ":" in val:
62                 # Handle MM:SS, HH:MM:SS, etc.
63                 parts = val.split(":")
64                 parts.reverse()
65                 total = 0.0
66                 for i, p in enumerate(parts):
67                     try:
68                         total += float(p) * (60 ** i)
69                     except ValueError:
70                         pass
71                 return total
72             try:
73                 return float(val)
74             except ValueError:
75                 logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76                 return 0.0
77                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78                 return 0.0
79
80     @staticmethod
81     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84         A highlight reel must never replay the same footage. The same rally can land in
85         the segment list more than once ? e.g. two detector runs accumulated for one
86         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
87         or
88         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
89         rally
90         twice. Merging any range that overlaps or abuts the previous one collapses true
91         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93         """
94         parsed = []
95         for raw_s, raw_e in segments:
96             s = VideoCompiler._parse_time(raw_s)
97             e = VideoCompiler._parse_time(raw_e)
98             if e > s:
99                 parsed.append((s, e))
100         parsed.sort()
101         merged: List[Tuple[float, float]] = []
102         for s, e in parsed:
103             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105             else:
106                 merged.append((s, e))
107         return merged
108
109     def _get_video_duration(self, video_path: str) -> float:
110         """Probes the video file to get its total duration in seconds.
111         Returns 0.0 if it cannot be determined."""

```

```

110         try:
111             result = subprocess.run(
112                 [
113                     "ffprobe", "-v", "error",
114                     "-show_entries", "format=duration",
115                     "-of", "default=noprint_wrappers=1:nokey=1",
116                     video_path
117                 ],
118                 capture_output=True, text=True, check=True
119             )
120             return float(result.stdout.strip())
121         except Exception as e:
122             logger.warning(f"Could not determine video duration via ffprobe: {e}")
123             return 0.0
124
125     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129         because the mark is already baked into every clip identically."""
130         wm = self.watermark
131         if not wm or not wm.get("enabled"):
132             return None
133
134         text = (wm.get("text") or "").strip()
135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
138             overlay.")
139             logo_path = ""
140         if not text and not logo_path:
141             return None
142
143         margin = int(wm.get("margin", 24))
144         opacity = float(wm.get("opacity", 0.85))
145         position = str(wm.get("position", "bottom-right"))
146         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
147         w/h (logo).
148         dt_xy = {
149             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
150             "bottom-left": f"x={margin}:y=h-th-{margin}",
151             "top-right": f"x=w-tw-{margin}:y={margin}",
152             "top-left": f"x={margin}:y={margin}",
153         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
154         ov_xy = {
155             "bottom-right": f"W-w-{margin}:H-h-{margin}",
156             "bottom-left": f"{margin}:H-h-{margin}",
157             "top-right": f"W-w-{margin}:{margin}",
158             "top-left": f"{margin}:{margin}",
159         }.get(position, f"W-w-{margin}:H-h-{margin}")
160
161         drawtext = ""
162         if text:
163             ratio = float(wm.get("font_size_ratio") or 0.035)
164             divisor = max(1, round(1.0 / ratio))
165             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
166             strings.
167             tf_path = os.path.join(tmp_dir, "watermark.txt")
168             with open(tf_path, "w", encoding="utf-8") as fh:
169                 fh.write(text)
170             # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
171             # without fontconfig) ? fontconfig family (last-resort, environment-
172             dependent).
173             font_path = (wm.get("font_path") or "").strip()
174             if not font_path and _BUNDLED_FONT.exists():

```

```

171         font_path = str(_BUNDLED_FONT)
172     if font_path:
173         font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174     else:
175         font_opt = f"font='{wm.get('font', 'Sans')}'"
176     box = ""
177     if wm.get("box", True):
178         box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179     drawtext = (
180         f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181         f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182     )
183
184     if logo_path and drawtext:
185         return
186     f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187     if logo_path:
188         return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189     return drawtext
190
191     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
192         """
193         Extracts selected segments from a raw video and stitches them together
194         into a seamless highlights file, applying end_padding to prevent abrupt endings.
195         """
196         if not segments:
197             raise ValueError("No highlight segments provided to compile.")
198
199         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
200         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
201         original_count = len(segments)
202         segments = self._merge_overlapping(segments)
203         if len(segments) < original_count:
204             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
205                 original_count - len(segments), original_count, len(segments))
206         if not segments:
207             raise ValueError("No valid (start < end) highlight segments after merge.")
208
209         if not os.path.exists(raw_video_path):
210             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
211
212         # Defense-in-depth: never let the output name escape output_dir (the API also
213         # validates this at the request boundary).
214         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
215
216         # Probe video duration so we can detect out-of-range segments
217         video_duration = self._get_video_duration(raw_video_path)
218         if video_duration > 0:
219             logger.info(f"Source video duration: {video_duration:.2f}s")
220         else:
221             logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
222
223         # We will create temporary clips and stitch them using FFmpeg concat demuxer
224         temp_clips = []
225         skipped_segments = []
226
227         # Create a temp directory within output_dir
228         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
229             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
230             # Build the (optional) watermark filtergraph once; it is applied during

```

```

every
230         # per-clip re-encode so the mark is baked into each clip identically.
231         watermark_filter = self._watermark_filter(tmp_dir)
232         if watermark_filter:
233             logger.info("[watermark] applying branding overlay to each clip.")
234         for idx, (raw_start, raw_end) in enumerate(segments):
235             # Normalize timestamps to float seconds ? handles float, int,
236             # plain numeric strings, and colon-separated formats like "MM:SS"
237             start = self._parse_time(raw_start)
238             end = self._parse_time(raw_end)
239             segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
240
241             # Validate the segment times
242             if start < 0:
243                 logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
244                 start = 0.0
245
246             if start >= end:
247                 logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
248                 skipped_segments.append((idx, start, end, "start >= end"))
249                 continue
250
251             # Check if segment extends beyond video duration
252             if video_duration > 0:
253                 if start >= video_duration:
254                     logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
255                                 f"This segment cannot be extracted ? the video ran
out. Skipping.")
256                     skipped_segments.append((idx, start, end, "start beyond video
end"))
257                     continue
258
259                 if end > video_duration:
260                     original_end = end
261                     end = video_duration
262                     logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
263                                 f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
264
265                 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
266
267             # Precise sub-second trim using fast re-encoding to preserve stream
headers
268             padded_start = max(0.0, start - self.begin_padding)
269             padded_end = end + self.end_padding
270
271             # Clamp padded_end to video duration if known
272             if video_duration > 0 and padded_end > video_duration:
273                 padded_end = video_duration
274                 logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
275                             f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
276
277             trim_duration = padded_end - padded_start
278             logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
279
280             base_cmd = [
281                 "ffmpeg", "-y",

```

```

282         "-ss", str(round(padded_start, 3)),
283         "-to", str(round(padded_end, 3)),
284         "-i", raw_video_path,
285         "-c:v", "libx264",
286         "-preset", "superfast",
287         "-crf", "22",
288         "-c:a", "aac",
289         "-b:a", "128k",
290     ]
291     vf_args = ["-vf", watermark_filter] if watermark_filter else []
292
293     # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
294     # path), retry once WITHOUT it so a branding misconfig can never nuke
the
295     # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
296     attempts = [vf_args, []] if vf_args else [[]]
297     produced = False
298     last_exc: Optional[subprocess.CalledProcessError] = None
299     cmd = base_cmd + [clip_path]
300     for attempt_vf in attempts:
301         cmd = base_cmd + attempt_vf + [clip_path]
302         try:
303             subprocess.run(cmd, capture_output=True, check=True)
304         except subprocess.CalledProcessError as e:
305             last_exc = e
306             if attempt_vf and vf_args:
307                 _err = (e.stderr.decode(errors="replace").strip()[-300:]
308                     if getattr(e, "stderr", None) else "")
309                 logger.warning(
310                     f"{segment_label}: watermark encode failed; retrying
without it. "
311                     f"ffmpeg: {_err or '(no stderr captured)}'"
312                 )
313                 continue
314             break
315         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
316             produced = True
317             if not attempt_vf and vf_args:
318                 logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")
319             break
320         last_exc = None # ffmpeg succeeded but produced nothing
321         break
322
323     if produced:
324         temp_clips.append(clip_path)
325     elif last_exc is not None:
326         stderr_msg = last_exc.stderr.decode(errors='replace').strip()
327         logger.error(f"{segment_label}: FFmpeg trim failed.")
328         print(f"  Command: {' '.join(cmd)}")
329         print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
330         skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
331         continue
332     else:
333         logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
334         skipped_segments.append((idx, start, end, "empty output clip"))
335         continue
336
337     # Report summary of skipped segments
338     if skipped_segments:

```

```

339         logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
340         for seg_idx, seg_start, seg_end, reason in skipped_segments:
341             logger.info(f"    - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
342             logger.info("")
343
344         if not temp_clips:
345             raise RuntimeError(
346                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
347                 f"All {len(segments)} segments were skipped. "
348                 f"Check the errors above for details. "
349                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
350             )
351
352         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
353
354         # Create a list file for FFmpeg concat demuxer
355         list_file_path = os.path.join(tmp_dir, "clips.txt")
356         with open(list_file_path, "w") as f:
357             for clip in temp_clips:
358                 # Escape single quotes in path
359                 safe_path = clip.replace("'", "'\\'")
360                 f.write(f"file '{safe_path}'\n")
361
362         logger.info("Stitching clips together...")
363         # Stitch them together without re-encoding (since they were encoded with
identical settings)
364         concat_cmd = [
365             "ffmpeg", "-y",
366             "-f", "concat",
367             "-safe", "0",
368             "-i", list_file_path,
369             "-c", "copy",
370             output_path
371         ]
372
373         try:
374             subprocess.run(concat_cmd, capture_output=True, check=True)
375
376             # Final verification
377             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
378                 raise RuntimeError(
379                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
380                     f"but output file is missing or empty: {output_path}"
381                 )
382
383             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
384             logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
385             return output_path
386         except subprocess.CalledProcessError as e:
387             stderr_msg = e.stderr.decode(errors='replace').strip()
388             raise RuntimeError(
389                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
390                 f"Command: {' '.join(concat_cmd)}\n"
391                 f"FFmpeg stderr: {stderr_msg[-500:]}"
392             ) from e
393

```

10. user (2026-06-18T07:54:53.039Z)

```

1      """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.

```

```

2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15      with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16      that feed the candidate `stats` JSON and the learned fusion features.
17
18    The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19    detector weights load only on first detection (`lazy_load_model`) ? so importing the
20    segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21    are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22    machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23    (ADR-005; ultralytics' AGPL YOLO was dropped).
24    """
25    import logging
26    from typing import Any, Callable, Dict, List, Optional, Tuple
27
28    import cv2
29    import torch
30
31    from backend.utils.geometry import get_velocity_normalised
32
33    logger = logging.getLogger(__name__)
34
35    # torchvision detection models are trained on COCO where the "person" category is
36    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38    _PERSON_CLASS_ID = 1
39
40    # Permissively-licensed torchvision detectors selectable via config
41    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42    _DETECTOR_FACTORIES = {
43        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46    }
47    _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50    class PersonDetector:
51        """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53        Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54        ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55        min_action_window_duration}``); the model loads lazily on first use.
56        """
57
58        def __init__(self, config: Dict[str, Any]):
59            self.config = config
60            # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61            # `self.model([tensor])` inference call type-checks without a quarantine.
62            self.model: Any = None

```

```

63         self.device: Optional[str] = None
64
65     def lazy_load_model(self) -> None:
66         if self.model is not None:
67             return
68
69         # Imported lazily so the module imports without torchvision present and so
70         # the test suite (which pre-sets self.model) never pulls in model weights.
71         from torchvision.models import detection as tv_detection
72
73         idx_cfg = self.config.get("indexing", {})
74         use_gpu = idx_cfg.get("use_gpu", True)
75         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76         if use_gpu and not torch.cuda.is_available():
77             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80         if model_name not in _DETECTOR_FACTORIES:
81             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82             model_name = _DEFAULT_DETECTOR
83
84         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87         self.model = builder(weights="DEFAULT")
88         self.model.eval()
89         self.model.to(self.device)
90
91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)
99
100     def detect_and_track(self, frame: Any, tracker: Any,
101                          score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's
107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
114         with torch.no_grad():
115             outputs = self.model([tensor])
116
117         out = outputs[0]
118         boxes = out["boxes"].detach().cpu().numpy()
119         labels = out["labels"].detach().cpu().numpy()
120         scores = out["scores"].detach().cpu().numpy()
121
122         # Keep only confident person detections.
123         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)

```

```

124         if not mask.any():
125             return []
126
127         # ByteTrack requires confidence to be populated (it filters on it internally).
128         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):
133             tid = tracked.tracker_id[i]
134             if tid is None:
135                 continue
136             x1, y1, x2, y2 = tracked.xyxy[i]
137             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138         return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                         velocity_thresh: float, merge_gap: float,
142                                         frame_skip: int = 3,
143                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
144         """
145         Runs person detection + tracking on a video chunk and extracts high-motion
146         action windows. Returns a list of dicts in the full video's timeframe, each
with:
147             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148             track_id_count, chunk_index.
149
150         Args:
151             frame_skip: Process every Nth frame (1 = every frame). Higher values
152                 dramatically reduce inference cost with minimal quality loss.
153             chunk_index: Index of the source chunk (recorded on each window for
traceability).
154         """
155         cap = cv2.VideoCapture(chunk_path)
156         if not cap.isOpened():
157             logger.error(f"Could not open {chunk_path}")
158             return []
159
160         fps = cap.get(cv2.CAP_PROP_FPS)
161         if fps <= 0:
162             fps = 30.0
163
164         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
165         if frame_width <= 0:
166             frame_width = 1280.0 # Sensible fallback
167
168         # Effective sampling rate after frame-skipping
169         effective_fps = fps / max(frame_skip, 1)
170
171         # A fresh tracker per chunk ? track IDs only need to be consistent within a
172         # chunk (windows are computed per chunk, then merged across chunks by time).
173         tracker = self.make_tracker(effective_fps)
174         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177         # so windows can be enriched with motion stats for logging + fine-tuning.
178         active_frames = []
179         frame_idx = 0
180         track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182         while True:
183             ret, frame = cap.read()

```

```

184         if not ret:
185             break
186
187         # Skip frames for performance ? at 30fps, every 3rd frame still
188         # gives ~10 samples/second, plenty for human-scale velocity.
189         if frame_idx % max(frame_skip, 1) != 0:
190             frame_idx += 1
191             continue
192
193         # Detect persons + assign persistent track IDs.
194         detections = self.detect_and_track(frame, tracker, score_thresh)
195
196         frame_peak_velocity = 0.0
197         frame_active_ids = set()
198         current_ids = set()
199
200         for track_id, bbox in detections:
201             current_ids.add(track_id)
202
203             if track_id in track_history:
204                 prev_bbox = track_history[track_id]
205                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207                 if velocity > velocity_thresh:
208                     frame_peak_velocity = max(frame_peak_velocity, velocity)
209                     frame_active_ids.add(track_id)
210
211                 track_history[track_id] = bbox
212
213         # Prune stale IDs that are no longer visible ? prevents false
214         # velocity spikes when the tracker reassigns an old ID.
215         track_history = {k: v for k, v in track_history.items() if k in current_ids}
216
217         if frame_active_ids:
218             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
219
220         frame_idx += 1
221
222     cap.release()
223
224     # Group active frames into continuous windows
225     if not active_frames:
226         return []
227
228     max_gap_frames = int(merge_gap * fps)
229
230     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
231         velocities = [v for _, v, _ in group]
232         track_ids = set().union(*[ids for _, _, ids in group])
233         return {
234             "start": group[0][0] / fps + chunk_start,
235             "end": group[-1][0] / fps + chunk_start,
236             "active_frame_count": len(group),
237             "peak_velocity": max(velocities),
238             "mean_velocity": sum(velocities) / len(velocities),
239             "track_id_count": len(track_ids),
240             "chunk_index": chunk_index,
241         }
242
243     windows = []
244     group = [active_frames[0]]
245     for af in active_frames[1:]:
246         if af[0] - group[-1][0] <= max_gap_frames:
247             group.append(af)
248         else:

```

```

248         windows.append(_finalize(group))
249         group = [af]
250     windows.append(_finalize(group))
251
252     # Filter out extremely short action windows
253     min_window_duration = self.config.get("indexing",
254     {}).get("min_action_window_duration", 2.0)
255     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
256
257     return windows
258
259 def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
260     frame_skip: int = 3) -> Dict[str, Any]:
261     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
262     video.
263     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
264     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
265     discards the
266     per-frame boxes), this returns the raw per-frame detections so the fusion
267     features can
268     align them with the shuttle trajectory. It reads the source by **absolute frame
269     index**
270     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
271     match
272     the WASB trajectory's ``.frame`` exactly ? that alignment is what
273     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
274     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
275     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
276     seek
277     accuracy varies by codec (it lands near the requested keyframe then decodes
278     forward);
279     treat the indices as ±a few frames on real footage ? fine for window-level
280     features.
281     """
282     self.lazy_load_model()
283     cap = cv2.VideoCapture(video_path)
284     if not cap.isOpened():
285         logger.error(f"Could not open {video_path}")
286         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
287
288     fps = cap.get(cv2.CAP_PROP_FPS)
289     if fps <= 0:
290         fps = 30.0
291     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
292     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
293     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
294     0.5)
295
296     step = max(frame_skip, 1)
297     tracker = self.make_tracker(fps / step)
298
299     start_frame = max(0, int(start_frame))
300     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
301     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
302     frame_idx = start_frame
303     while frame_idx <= end_frame:
304         ret, frame = cap.read()
305         if not ret:
306             break
307         if (frame_idx - start_frame) % step == 0:
308             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
309             frame_idx += 1
310     cap.release()
311     return {"frames": frames, "fps": fps,

```

```

303         "frame_width": float(frame_width), "frame_height": float(frame_height)}
304
305     def detections_over_windows(self, video_path: str, windows: List[Tuple[int, int]],
306                               frame_skip: int = 3,
307                               progress_cb: Optional[Callable[[int, int], None]] = None
308                               ) -> List[Dict[str, Any]]:
309         """Per-window person detections via ONE sequential pass over the video.
310
311         Opens the video ONCE and reads forward ? ``grab()`` to advance, ``retrieve()`` +
312         detect only on sampled frames inside a window ? instead of
313         ``detections_per_frame``'s
314         per-window open + ``POS_FRAMES`` seek, which on a long video degrades to
315         decode-from-start per window (0(window_index) ? the fusion-P3 extraction
316         bottleneck;
317         172 windows took ~80 min). ``windows`` = ``[(start_frame, end_frame), ...]``;
318         returns a
319         list aligned to it, each ``{"frames": {idx: [(tid, bbox)]}, "fps", "frame_width",
320         "frame_height"}``` (same shape as ``detections_per_frame``). Each window gets its
321         own
322         window-local ByteTrack. ``progress_cb(frames_done, total_frames)`` (called every
323         ~256
324         frames) drives progress logging.
325         """
326         self.lazy_load_model()
327         cap = cv2.VideoCapture(video_path)
328         n = len(windows)
329         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
330         if fps <= 0:
331             fps = 30.0
332         fw = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
333         fh = cap.get(cv2.CAP_PROP_FRAME_HEIGHT) if cap.isOpened() else 0.0
334         base = {"fps": fps, "frame_width": float(fw), "frame_height": float(fh)}
335         if not cap.isOpened() or n == 0:
336             cap.release()
337             return [dict(frames={}, **base) for _ in range(n)]
338
339         step = max(frame_skip, 1)
340         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
341         0.5)
342         frames_per_win: List[Dict[int, Any]] = [{ } for _ in range(n)]
343         trackers = [self.make_tracker(fps / step) for _ in range(n)]
344         max_end = max(ef for _, ef in windows)
345
346         frame_idx = 0
347         while frame_idx <= max_end:
348             if not cap.grab():
349                 break
350             # Windows that want THIS frame (inside the range AND on that window's
351             sampling grid).
352             wis = [wi for wi, (sf, ef) in enumerate(windows)
353                   if sf <= frame_idx <= ef and (frame_idx - sf) % step == 0]
354             if wis:
355                 ok, frame = cap.retrieve()
356                 if ok:
357                     for wi in wis:
358                         frames_per_win[wi][frame_idx] = self.detect_and_track(
359                             frame, trackers[wi], score_thresh)
360             frame_idx += 1
361             if progress_cb is not None and frame_idx % 256 == 0:
362                 progress_cb(frame_idx, max_end)
363         cap.release()
364         return [dict(frames=frames_per_win[wi], **base) for wi in range(n)]
365
366

```

Now let me find the test files for each of these.

12. user (2026-06-18T07:54:56.829Z)

```
tests\test_calibrate_wasb.py
tests\test_config_wasb.py
tests\test_export_wasb_segments.py
tests\test_wasb_cache.py
tests\test_wasb_runner.py
```

13. user (2026-06-18T07:54:57.307Z)

```
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
```

14. user (2026-06-18T07:54:57.784Z)

```
tests\test_person_detector.py
tests\test_personalization_serving.py
```

15. user (2026-06-18T07:54:57.915Z)

```
tests\test_compiler.py
```

16. assistant (2026-06-18T07:55:00.453Z)

Let me also look for the wasb_infer streaming test, and read each test file to assess coverage.

17. user (2026-06-18T07:55:00.958Z)

No files found

18. user (2026-06-18T07:55:01.294Z)

No files found

19. user (2026-06-18T07:55:03.209Z)

```
Found 9 files
tests\test_windowing.py
tests\test_wsl_tilde_quoting.py
tests\test_wasb_cache.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_served_gate_a_regression.py
tests\test_fusion_hybrid.py
tests\test_detector_keying.py
```

20. user (2026-06-18T07:55:05.539Z)

```
1      """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3      Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4      detection accumulation on replay, crash-truncation recovery, manifest
5      compatibility/invalidation, and atomic trajectory writes.
6
7      Also covers the streaming fast-path's pure helpers (synthetic frame addressing,
8      window builder, the sliding-buffer frame store, and the read_image shim) ? these
9      let the detector read frames straight from the decoded video instead of from ~46k
10     PNGs on disk, while preserving output exactly (byte-identical, verified).
11     """
12     import json
```

```

13 import os
14 import os.path as osp
15
16 import numpy as np
17 import pytest
18
19 from backend.pipeline.detectors import wasb_infer as wi
20
21
22 def _win(idx, frames):
23     """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
24     return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
25
26
27 def _write_windows(cache_dir, windows):
28     det_jsonl, _ = wi._cache_paths(cache_dir)
29     with open(det_jsonl, "a") as f:
30         wi._append_windows(f, windows)
31
32
33 # ----- #
34 # detection (de)serialization
35 # ----- #
36 def test_det_plain_and_back_roundtrip():
37     det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
38     plain = wi._det_plain(det)
39     assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
40
41     back = wi._dets_to_tracker([plain])
42     assert len(back) == 1
43     assert isinstance(back[0]["xy"], np.ndarray) # tracker does vector math on xy
44     assert back[0]["xy"].tolist() == [439.0, 64.5]
45     assert back[0]["scale"] == 0
46
47
48 # ----- #
49 # cache replay + resume index
50 # ----- #
51 def test_replay_accumulates_detections_across_windows(tmp_path):
52     cache = str(tmp_path / "c")
53     os.makedirs(cache)
54     # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
55     # frame 2 is empty in window 0 but detected in window 1.
56     _write_windows(cache, [
57         _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
58         _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
59     ])
60     windows_done, det = wi.load_and_clean_cache(cache)
61     assert windows_done == 2
62     assert det[0] == [[10, 20, 0.9, 0]]
63     assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]] # window-0 then window-1
64     assert det[2] == [[13, 23, 0.6, 0]]
65     assert det[3] == [] # seen but never detected
66
67
68 def test_resume_index_equals_window_count(tmp_path):
69     cache = str(tmp_path / "c")
70     os.makedirs(cache)
71     _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
72     windows_done, _ = wi.load_and_clean_cache(cache)
73     assert windows_done == 5
74
75
76 def test_empty_cache(tmp_path):
77     cache = str(tmp_path / "c")

```

```

78     os.makedirs(cache)
79     windows_done, det = wi.load_and_clean_cache(cache)
80     assert windows_done == 0 and dict(det) == {}
81
82
83     # ----- #
84     # crash robustness
85     # ----- #
86     def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
87         cache = str(tmp_path / "c")
88         os.makedirs(cache)
89         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
90                               _win(1, [(1, [[2, 2, 0.8, 0]])]))
91         det_jsonl, _ = wi._cache_paths(cache)
92         with open(det_jsonl, "a") as f: # simulate a crash mid-flush
93             f.write('{"w":2,"f":[[2,[[3,3,')
94         windows_done, det = wi.load_and_clean_cache(cache)
95         assert windows_done == 2 # only the two clean windows
survive
96         assert det[1] == [[2, 2, 0.8, 0]]
97         # file is rewritten to the clean prefix -> appending window 2 stays contiguous
98         with open(det_jsonl) as f:
99             lines = [json.loads(ln) for ln in f if ln.strip()]
100         assert [o["w"] for o in lines] == [0, 1]
101         _write_windows(cache, [_win(2, [(2, [[3, 3, 0.7, 0]])]))
102         assert wi.load_and_clean_cache(cache)[0] == 3
103
104
105     def test_noncontiguous_window_index_stops_replay(tmp_path):
106         cache = str(tmp_path / "c")
107         os.makedirs(cache)
108         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
109                               _win(5, [(5, [[2, 2, 0.8, 0]])])) # gap -> stop after w=0
110         windows_done, det = wi.load_and_clean_cache(cache)
111         assert windows_done == 1
112         assert 5 not in det
113
114
115     # ----- #
116     # manifest: atomic write + compatibility / invalidation
117     # ----- #
118     def test_manifest_roundtrip_and_compat(tmp_path):
119         cache = str(tmp_path / "c")
120         os.makedirs(cache)
121         m = {
122             "version": wi.CACHE_VERSION,
123             "frames_dir": osp.abspath("/x/frames"),
124             "weights": "w.pth.tar",
125             "sport": "badminton",
126             "frames_in": 3,
127             "frames_total": 100,
128             "windows_total": 98,
129             "windows_done": 42,
130         }
131         wi.write_manifest(cache, m)
132         assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp")) # atomic, no
tmp left
133         assert wi.read_manifest(cache) == m
134
135         base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
136                     frames_in=3, frames_total=100)
137         assert wi.manifest_compatible(m, **base)
138         assert not wi.manifest_compatible(m, **{**base, "frames_total": 101}) # more
frames
139         assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})

```

```

140     assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
141     assert not wi.manifest_compatible({}, **base) # no cache
142     assert not wi.manifest_compatible(**m, "version": 999}, **base) # schema bump
143
144
145 def test_read_manifest_missing_returns_none(tmp_path):
146     assert wi.read_manifest(str(tmp_path)) is None
147
148
149 def test_reset_cache_removes_files(tmp_path):
150     cache = str(tmp_path / "c")
151     os.makedirs(cache)
152     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]]))]])
153     wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
154     wi.reset_cache(cache)
155     det_jsonl, mpath = wi._cache_paths(cache)
156     assert not osp.exists(det_jsonl) and not osp.exists(mpath)
157
158
159 # ----- #
160 # misc helpers
161 # ----- #
162
163 def test_default_cache_dir_is_next_to_output():
164     out = osp.join("some", "dir", "sample_long_traj.csv")
165     cd = wi.default_cache_dir(out)
166     assert osp.basename(cd) == "sample_long_traj_wasbcache"
167     assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
168
169
170 def test_atomic_write_trajectory(tmp_path):
171     out = str(tmp_path / "out" / "traj.csv")
172     wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
173     assert not osp.exists(out + ".tmp")
174     with open(out) as f:
175         text = f.read().splitlines()
176     assert text[0] == "Frame,Visibility,X,Y"
177     assert text[1] == "0,1,439.0,64.0"
178     assert text[2] == "1,0,-1,-1"
179
180
181 # ----- #
182 # streaming fast-path: synthetic frame addressing + windowing (pure)
183 # ----- #
184
185 def test_synthetic_frame_path_roundtrip():
186     # The synthetic name must match the on-disk scheme so _frame_id parses the SAME id.
187     assert wi.synthetic_frame_path(0) == "00000000.png"
188     assert wi.synthetic_frame_path(180) == "00000180.png"
189     for i in (0, 1, 7, 180, 46123):
190         assert wi._index_from_synthetic_path(wi.synthetic_frame_path(i)) == i
191         # disk-style names parse to the same id (output-preservation invariant)
192         assert wi._frame_id(f"frame_{i:08d}.png") == i
193
194
195 def test_build_windows_matches_disk_logic():
196     frames = [wi.synthetic_frame_path(i) for i in range(6)]
197     # frames_in=3 -> 4 sliding windows, each 3 consecutive, overlapping by 2.
198     wins = wi.build_windows(frames, 3)
199     assert len(wins) == 4
200     assert wins[0] == frames[0:3]
201     assert wins[1] == frames[1:4]
202     assert wins[-1] == frames[3:6]
203
204
205 def test_build_windows_too_few_frames_is_empty():
206     assert wi.build_windows(["00000000.png", "00000001.png"], 3) == []

```

```

205
206
207 # ----- #
208 # streaming fast-path: SequentialFrameStore (sliding buffer; pure)
209 # ----- #
210 def _counting_reader():
211     """A fake forward-only decoder: returns ('frame', i) and records call order."""
212     calls = []
213
214     def reader(i):
215         calls.append(i)
216         return ("frame", i)
217
218     return reader, calls
219
220
221 def test_store_reads_each_frame_once_in_order_for_sliding_windows():
222     # The detector's real access pattern: ImageDataset reads window i's frames in order
223     # (i, i+1, i+2), and samples are requested i=0,1,2,... So the global read sequence
224     # DIPS BACK by window-1 at each boundary: 0,1,2, 1,2,3, 2,3,4. Every source frame
225     # must still be decoded exactly once (the whole point of the in-memory buffer).
226     reader, calls = _counting_reader()
227     store = wi.SequentialFrameStore(reader, total=5, window=3)
228     order = [0, 1, 2, 1, 2, 3, 2, 3, 4]
229     got = [store.get(i) for i in order]
230     assert got == [("frame", i) for i in order]
231     assert calls == [0, 1, 2, 3, 4] # decoded once each, in order
232
233
234 def test_store_evicts_outside_window_bounded_memory():
235     # window=3 -> buffer holds at most 3 frames; frames older than max_seen-2 are
dropped.
236     reader, _ = _counting_reader()
237     store = wi.SequentialFrameStore(reader, total=100, window=3)
238     for i in (0, 1, 2, 3, 4):
239         store.get(i)
240     assert len(store._buf) <= 3
241     # frame 1 is now below the floor (max_seen=4, floor=2) -> evicted, re-request
raises.
242     with pytest.raises(KeyError):
243         store.get(1)
244
245
246 def test_store_keeps_window_minus_one_lookback():
247     # The legal dip-back of exactly window-1 must NOT raise (it's the real access
pattern).
248     reader, _ = _counting_reader()
249     store = wi.SequentialFrameStore(reader, total=10, window=3)
250     store.get(0)
251     store.get(1)
252     store.get(2) # max_seen=2, floor=0 -> frames 0,1,2 all retained
253     assert store.get(1) == ("frame", 1) # window boundary look-back is fine
254     assert store.get(2) == ("frame", 2)
255
256
257 def test_store_resume_starts_at_start_index_without_decoding_prefix():
258     # On a resume the first needed frame is `start_index`; the prefix must NOT be
decoded.
259     reader, calls = _counting_reader()
260     store = wi.SequentialFrameStore(reader, total=10, window=3, start_index=4)
261     store.get(4)
262     store.get(5)
263     assert calls == [4, 5] # frames 0..3 skipped entirely (seek, not decode)
264     with pytest.raises(KeyError): # below the resume floor -> never available
265         store.get(3)

```

```

266
267
268 def test_store_out_of_range_raises():
269     reader, _ = _counting_reader()
270     store = wi.SequentialFrameStore(reader, total=3, window=3)
271     with pytest.raises(KeyError):
272         store.get(3)
273
274
275 # ----- #
276 # streaming fast-path: read_image shim resolves synthetic paths to in-memory RGB
277 # frames (WASB reads via PIL read_image, NOT cv2.imread) + falls through otherwise.
278 # ----- #
279 def _bgr_frame(b, g, r, h=4, w=4):
280     cv2 = pytest.importorskip("cv2") # noqa: F841 (importorskip gates the suite on cv2)
281     frame = np.zeros((h, w, 3), dtype=np.uint8)
282     frame[..., 0], frame[..., 1], frame[..., 2] = b, g, r
283     return frame
284
285
286 def test_make_streamed_read_image_returns_pil_rgb_and_falls_through():
287     cv2 = pytest.importorskip("cv2")
288
289     # Loader yields a distinct-channel BGR frame per index (B=10+i, G=20+i, R=30+i).
290     def loader(i):
291         return _bgr_frame(10 + i, 20 + i, 30 + i)
292
293     store = wi.SequentialFrameStore(loader, total=4, window=1)
294     shim = wi._make_streamed_read_image(store, real_read_image=lambda p, *a, **k:
295 ("REAL", p))
296
297     img = shim(wi.synthetic_frame_path(0))
298     arr = np.asarray(img)
299     # The shim must return an RGB array (channels swapped from the BGR loader frame),
300 matching
301     # what WASB's read_image (PIL Image.open().convert('RGB')) yields off disk ? NOT raw
302 BGR.
303     assert arr.shape == (4, 4, 3)
304     assert (arr[..., 0] == 30).all() # R
305     assert (arr[..., 1] == 20).all() # G
306     assert (arr[..., 2] == 10).all() # B
307     assert np.array_equal(arr, cv2.cvtColor(loader(0), cv2.COLOR_BGR2RGB))
308
309     # A path with no parseable index falls through to the real reader.
310     assert shim("no-digits-here") == ("REAL", "no-digits-here")
311
312
313 def test_make_streamed_read_image_resume_primes_start_index():
314     pytest.importorskip("cv2")
315     calls = []
316
317     def loader(i):
318         calls.append(i)
319         return _bgr_frame(1, 2, 3)
320
321     store = wi.SequentialFrameStore(loader, total=10, window=1, start_index=5)
322     shim = wi._make_streamed_read_image(store, real_read_image=lambda p: None)
323     shim(wi.synthetic_frame_path(5))
324     assert calls == [5] # resume seeked to 5; frames 0..4 never decoded
325
326
327 def test_bgr_to_pil_rgb_matches_disk_roundtrip(tmp_path):
328     """Regression lock on the output-preservation proof: the streaming frame conversion
329     (`_bgr_to_pil_rgb`) is byte-identical to WASB's disk path
330     (`cv2.imread` -> `Image.open().convert('RGB')`)."""

```

```

328     cv2 = pytest.importorskip("cv2")
329     Image = pytest.importorskip("PIL.Image")
330
331     rng = np.random.RandomState(0)
332     frame_bgr = rng.randint(0, 256, size=(16, 24, 3), dtype=np.uint8)
333
334     # Disk path: BGR -> PNG (lossless) -> PIL RGB
335     png = osp.join(str(tmp_path), "f.png")
336     cv2.imwrite(png, frame_bgr)
337     disk_rgb = np.asarray(Image.open(png).convert("RGB"))
338
339     # Streaming path: BGR -> PIL RGB directly
340     stream_rgb = np.asarray(wi._bgr_to_pil_rgb(frame_bgr))
341
342     assert np.array_equal(disk_rgb, stream_rgb)
343

```

21. assistant (2026-06-18T07:55:09.036Z)

Strong coverage on wasb_infer's pure helpers. The `run()` function itself (the 185-line GPU orchestration) is NOT directly tested ? it imports torch/WASB modules. Now let me check the tracknet and windowing tests, and the compiler test.

22. user (2026-06-18T07:55:09.906Z)

```

1     """Tests for backend.eval.windowing ? the single-sourced SERVED/PROPOSAL presets.
2
3     GPU-free. Pins the foundational invariants:
4     - the SERVED preset is the SAME windowing production serves (sourced from the config
5       defaults, never a hand-typed tuple that can drift);
6     - PROPOSAL and SERVED are genuinely DIFFERENT operating points ? different candidate
sets;
7     - distill_local's module-level presets stay byte-identical to the legacy tuples;
8     - served_windowing_from_config reproduces process_video's knob reads (overrides
honoured).
9     """
10    import csv
11
12    from backend.config.models import AppConfig, IndexingConfig
13    from backend.eval import distill_local as dl
14    from backend.eval import windowing as w
15    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
16
17
18    def _write_traj(path):
19        """A trajectory with a SHORT fast burst (~0.8s) then, after a gap, a LONG fast rally
(~5s),
20        at fps=10. The short burst clears the permissive PROPOSAL min_window (0.5s) but is
dropped by
21        the coarse SERVED one (2.0s), so the two presets yield a different number of windows
? the
22        eval?serve fact in miniature."""
23        rows = [("Frame", "Visibility", "X", "Y")]
24        # 0-8: short fast burst (~0.8s of motion) ? survives proposal, dropped by served
(min 2s).
25        x = 50
26        for i in range(0, 8):
27            x += 40
28            rows.append((i, 1, x, 50))
29        # 8-30: occluded (a long gap so the burst and the rally never merge into one
window).
30        for i in range(8, 30):
31            rows.append((i, 0, -1, -1))
32        # 30-80: long fast rally (~5s of motion) ? survives BOTH presets.
33        x = 50

```

```

34         for i in range(30, 80):
35             x += 40
36             rows.append((i, 1, x, 50))
37         with open(path, "w", newline="") as f:
38             csv.writer(f).writerows(rows)
39         return str(path)
40
41
42     def test_served_preset_matches_config_defaults():
43         """SERVED is single-sourced from IndexingConfig defaults ? exactly what
process_video reads."""
44         idx = IndexingConfig()
45         assert W.SERVED.velocity_thresh == idx.wasb.velocity_threshold
46         assert W.SERVED.merge_gap == idx.dead_time_merge_gap
47         assert W.SERVED.min_window_duration == idx.min_action_window_duration
48         # The historically-served operating point.
49         assert W.SERVED.as_tuple() == (0.02, 2.0, 2.0)
50
51
52     def test_proposal_preset_is_the_recall_oriented_windowing():
53         assert W.PROPOSAL.as_tuple() == (0.005, 1.0, 0.5)
54         assert W.PRESETS == {"served": W.SERVED, "proposal": W.PROPOSAL}
55
56
57     def test_distill_local_presets_byte_identical_to_legacy_tuples():
58         """The eval substrate / golden JSONs must stay valid ? presets unchanged in
value."""
59         assert dl.PROPOSAL == (0.005, 1.0, 0.5)
60         assert dl.BASELINE_LOCAL == (0.02, 2.0, 2.0)
61         assert dl.PROPOSAL == W.PROPOSAL.as_tuple()
62         assert dl.BASELINE_LOCAL == W.SERVED.as_tuple()
63
64
65     def test_served_and_proposal_produce_different_candidates(tmp_path):
66         """The core eval?serve fact: the two windowings window the SAME trajectory
differently."""
67         points = TrackNetRunner.parse_trajectory_csv(write_traj(tmp_path / "t.csv"))
68         proposal_w = W.build_windows(points, fps=10.0, frame_width=100.0, preset=W.PROPOSAL)
69         served_w = W.build_windows(points, fps=10.0, frame_width=100.0, preset=W.SERVED)
70         p_iv = W.windows_to_intervals(proposal_w)
71         s_iv = W.windows_to_intervals(served_w)
72         # Permissive proposal finds at least as many windows as the coarse served one ...
73         assert len(p_iv) >= len(s_iv)
74         # ... and the two candidate SETS genuinely differ (so a cue scored on one ? the
other).
75         assert p_iv != s_iv
76
77
78     def test_served_windowing_from_typed_config():
79         cfg = AppConfig()
80         for fam in ("wasb", "fusion", "tracknet"):
81             assert W.served_windowing_from_config(cfg, fam).as_tuple() == (0.02, 2.0, 2.0)
82
83
84     def test_served_windowing_from_config_honours_overrides():
85         """A deployment that tuned a served default ? served_windowing_from_config follows
it."""
86         d = {"indexing": {"wasb": {"velocity_threshold": 0.03,
87                                     "dead_time_merge_gap": 1.5,
88                                     "min_action_window_duration": 2.5}}}
89         assert W.served_windowing_from_config(d, "wasb").as_tuple() == (0.03, 1.5, 2.5)
90
91
92     def test_served_windowing_from_config_falls_back_to_defaults_on_missing_keys():
93         assert W.served_windowing_from_config({}, "wasb").as_tuple() == W.SERVED.as_tuple()

```

```

94         assert W.served_windowing_from_config(None, "wasb").as_tuple() ==
W.SERVED.as_tuple()
95
96
97     def test_served_windowing_from_config_honors_r5_blob_fallback():
98         # The R5 blob-fallback + its magnitude gate lift out of a live config (so the
eventual default
99         # flip and any per-deployment override take effect on the served path).
100         d = {"indexing": {"window_blob_fallback": True, "window_blob_factor": 5.0}}
101         p = W.served_windowing_from_config(d, "wasb")
102         assert p.blob_fallback is True and p.blob_factor == 5.0
103
104
105     def test_preset_roundtrip_dict():
106         # R1 milestone FLIPPED ON (2026-06-18): the SERVED defaults now carry W1 cap=6 + R4.
107         d = W.SERVED.to_dict()
108         assert d == {"name": "served", "velocity_thresh": 0.02, "merge_gap": 2.0,
109                     "min_window_duration": 2.0, "max_window_duration": 6.0, "hard_cap":
False,
110                     "inactivity_end": 1.5, "inactivity_rally_thresh": 0.20,
"inactivity_min_density": 0.30,
111                     "blob_fallback": False, "blob_factor": 4.0}
112
113
114     def test_served_preset_is_r1_milestone_default():
115         # R1 milestone FLIPPED ON (2026-06-18, PR #204): W1 bounded windowing (cap=6) + the
R4
116         # rally-END inactivity trim are now the shipped SERVED defaults (the smallest safe
quality
117         # flip, n=13 baseline?milestone ?tolF1 +0.222, sign 12/0, p=0.0005). The W1.5 hard-
cap, the
118         # R5 blob-fallback, and W2 net-crossings all stay OFF.
119         assert W.SERVED.max_window_duration == 6.0
120         assert W.SERVED.inactivity_end == 1.5
121         assert W.SERVED.inactivity_rally_thresh == 0.20
122         assert W.SERVED.inactivity_min_density == 0.30
123         assert W.SERVED.hard_cap is False          # W1.5 hard-cap stays OFF
124         assert W.SERVED.blob_fallback is False      # R5 blob-fallback stays OFF
125         # served_windowing_from_config follows a live config; absent keys fall back to the
SERVED
126         # (now-milestone) defaults, and explicit overrides still win.
127         assert W.served_windowing_from_config({}, "wasb").max_window_duration == 6.0
128         assert W.served_windowing_from_config({"indexing": {"window_hard_cap": True}},
"wasb").hard_cap is True
129         assert W.served_windowing_from_config({"indexing": {"max_window_duration": 0.0}},
"wasb").max_window_duration == 0.0
130

```

23. assistant (2026-06-18T07:55:12.411Z)

Now let me look at the tracknet_runner test and the windowing module to understand how `trajectory\_to\_action\_windows` is tested.

24. user (2026-06-18T07:55:13.060Z)

```

1      """Tests for the TrackNet runner's pure-Python pieces (path translation, CSV
2      parsing, trajectory -> action-window conversion). The WSL subprocess invocation
3      itself is not exercised here ? these run anywhere, no WSL/TensorFlow needed."""
4
5      import os
6      import subprocess
7      import textwrap
8      from unittest.mock import patch
9
10     from backend.pipeline.detectors.tracknet_runner import (

```

```

11     TrackNetRunner,
12     TrackNetConfig,
13     TrajectoryPoint,
14     to_wsl_mnt_path,
15 )
16 from backend.pipeline.detectors.base import DetectorRunner
17 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
18
19
20 def _proc(returncode=0, stdout="", stderr=""):
21     return subprocess.CompletedProcess(args=[], returncode=returncode,
22                                         stdout=stdout, stderr=stderr)
23
24
25 def test_to_wsl_mnt_path_windows():
26     assert to_wsl_mnt_path(r"C:\Users\avidu\v.mp4") == "/mnt/c/Users/avidu/v.mp4"
27     assert to_wsl_mnt_path(r"D:\clips\m.mp4") == "/mnt/d/clips/m.mp4"
28
29
30 def test_to_wsl_mnt_path_passthrough():
31     # Already-POSIX paths are left alone.
32     assert to_wsl_mnt_path("/home/avidullu/clips/v.mp4") == "/home/avidullu/clips/v.mp4"
33     assert to_wsl_mnt_path("~/clips/v.mp4") == "~/clips/v.mp4"
34
35
36 def test_config_from_indexing_cfg():
37     cfg = TrackNetConfig.from_indexing_cfg({"tracknet": {
38         "wsl_distro": "Ubuntu-22.04", "conda_env": "TN", "weights_path": "/w/m.h5",
39         "queue_length": 7, "stage_in_wsl": False,
40     }})
41     assert cfg.distro == "Ubuntu-22.04"
42     assert cfg.conda_env == "TN"
43     assert cfg.weights_path == "/w/m.h5"
44     assert cfg.queue_length == 7
45     assert cfg.stage_in_wsl is False
46
47
48 def test_config_defaults_when_absent():
49     cfg = TrackNetConfig.from_indexing_cfg({})
50     assert cfg.distro == "Ubuntu"
51     assert cfg.conda_env == "TrackNetV4"
52     assert cfg.weights_path == "" # must be supplied by the user
53
54
55 def test_parse_trajectory_csv(tmp_path):
56     csv_file = tmp_path / "v_predict.csv"
57     csv_file.write_text(textwrap.dedent("""\
58         Frame,Visibility,X,Y
59         0,0,0,0
60         1,1,100,200
61         2,1,140,210
62         3,0,0,0
63     """))
64     pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
65     assert len(pts) == 4
66     assert pts[0].visible is False
67     assert pts[1].visible is True and pts[1].x == 100 and pts[1].y == 200
68     assert pts[3].visible is False
69
70
71 def test_trajectory_to_action_windows_detects_moving_shuttle():
72     # 30 fps, 1280px wide. Shuttle moves ~60px/frame while visible (frames 0..29) =>
73     # well above the velocity threshold; then disappears. Expect one window ~1s long.
74     fps, width = 30.0, 1280.0
75     pts = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in

```

```

range(30)]
76     pts += [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(30, 60)]
77
78     windows = TrackNetRunner.trajectory_to_action_windows(
79         pts, fps=fps, frame_width=width, velocity_thresh=0.02,
80         merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
81     )
82     assert len(windows) == 1
83     w = windows[0]
84     assert w["start"] < w["end"]
85     assert w["active_frame_count"] > 0
86     assert w["peak_velocity"] > 0.02
87     assert w["chunk_index"] == 0
88
89
90     def test_trajectory_to_action_windows_ignores_stationary_shuttle():
91         # Visible but barely moving (1px/frame) => below threshold => no windows.
92         pts = [TrajectoryPoint(frame=i, visible=True, x=500 + i, y=300) for i in range(60)]
93         windows = TrackNetRunner.trajectory_to_action_windows(
94             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.05,
95             merge_gap=2.0, min_window_duration=0.5,
96         )
97         assert windows == []
98
99
100    def test_trajectory_to_action_windows_splits_on_gap():
101        # Two bursts of motion separated by a long invisible gap => two windows.
102        burst1 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
range(20)]
103        gap = [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(20, 140)]
104        burst2 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - 140), y=300)
105                  for i in range(140, 160)]
106        windows = TrackNetRunner.trajectory_to_action_windows(
107            burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
108            velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
109        )
110        assert len(windows) == 2
111        assert windows[0]["end"] < windows[1]["start"]
112
113
114    # ---- W1: bounded windowing (max_window_duration split)
----- #
115    def _moving(a, b): # visible, fast-moving shuttle over frames [a, b)
116        return [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - a), y=300) for i in
range(a, b)]
117
118
119    def _invisible(a, b):
120        return [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(a, b)]
121
122
123    # Two rallies merged across a sub-merge_gap dead time ? ONE over-long window without the
cap.
124    _MERGED PTS = _moving(0, 121) + _invisible(121, 162) + _moving(162, 243) # ~8s, 1.4s
internal gap
125
126
127    def test_max_window_duration_off_keeps_over_merge():
128        # Default (max_window_duration=0): the two rallies stay merged into one window.
129        w = TrackNetRunner.trajectory_to_action_windows(
130            _MERGED PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
131            merge_gap=2.0, min_window_duration=0.3)
132        assert len(w) == 1 # over-merge reproduced (the failure the cap fixes)
133        assert (w[0]["end"] - w[0]["start"]) > 7.0
134

```

```

135
136     def test_max_window_duration_splits_at_inactivity_gap():
137         # With a 4s cap the 8s merged window is split at its 1.4s internal dead-time gap ? 2
138         windows.
139         w = TrackNetRunner.trajectory_to_action_windows(
140             _MERGED_PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
141             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
142             min_split_gap=0.5)
143         assert len(w) == 2
144         assert w[0]["end"] < w[1]["start"]
145         assert all((x["end"] - x["start"]) <= 5.0 for x in w)
146
147     def test_max_window_duration_preserves_genuine_long_rally():
148         # A single continuous 10s rally (no internal gap ? min_split_gap) is NOT split, even
149         # though it exceeds the cap ? bounded windowing only cuts at real dead time.
150         w = TrackNetRunner.trajectory_to_action_windows(
151             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
152             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
153             min_split_gap=0.5)
154         assert len(w) == 1
155         assert (w[0]["end"] - w[0]["start"]) > 9.0
156
157     def test_split_overlong_groups_pure():
158         # Direct check of the splitter: a group with one big internal gap splits there.
159         grp = [(i, 1.0) for i in range(0, 20)] + [(i, 1.0) for i in range(60, 80)] # gap
160         out = TrackNetRunner._split_overlong_groups([grp], max_win_frames=30,
161             min_split_frames=10)
162         assert len(out) == 2 and out[0][-1][0] == 19 and out[1][0][0] == 60
163         # No qualifying gap (all consecutive) ? unchanged even when over length.
164         solid = [(i, 1.0) for i in range(0, 100)]
165         assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
166
167     # ---- W1 hard-cap fallback: enforce the cap on continuously-active trajectories
168     ----- #
169     def test_window_hard_cap_chops_continuous_rally():
170         # The same continuous ~10s rally that bounded-windowing alone LEAVES intact (no
171         # gap) is, with window_hard_cap=True, chopped at the midpoint until every piece ?
172         the cap.
173         w = TrackNetRunner.trajectory_to_action_windows(
174             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
175             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
176             min_split_gap=0.5,
177             window_hard_cap=True)
178         assert len(w) >= 3 # the 10s blob is broken
179         up
180         assert all((x["end"] - x["start"]) <= 4.01 for x in w) # every piece within the
181         cap
182         assert w[0]["start"] <= 0.1 and w[-1]["end"] >= 9.0 # coverage preserved
183         (only cuts)
184
185     def test_split_overlong_groups_hard_cap_pure():
186         solid = [(i, 1.0) for i in range(0, 100)] # 99-frame span, no
187         internal gap
188         # default: kept whole; hard_cap: chopped into pieces each spanning ? max_win_frames
189         assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
190         out = TrackNetRunner._split_overlong_groups([solid], 30, 10, hard_cap=True)
191         assert len(out) >= 4
192         assert all((g[-1][0] - g[0][0]) <= 30 for g in out)
193         # only cuts, never drops/duplicates: the pieces partition the original frames

```

```

exactly
187         assert sorted(f for g in out for f, _ in g) == list(range(0, 100))
188
189
190     # ---- R4: rally-END inactivity trim (drop the post-rally slow-drift tail)
----- #
191     def test_trim_inactive_tail_drops_slow_drift():
192         # fast rally (vel 0.5) frames 0..60, then slow drift (vel 0.03, still > active 0.02)
61..200.
193         group = [(i, 0.5) for i in range(0, 61)] + [(i, 0.03) for i in range(61, 201)]
194         out = TrackNetRunner._trim_inactive_tail(group, trail_frames=30, min_density=0.3,
rally_thresh=0.2)
195         assert out[0][0] == 0                                # start untouched
196         assert out[-1][0] < 100                             # the pure-drift tail (frames
100..200) is trimmed off
197         assert len(group) - len(out) >= 100                 # ~all the slow drift cut (kept only a
short decay tail)
198
199
200     def test_trim_inactive_tail_keeps_continuous_rally_and_is_noop_when_off():
201         allfast = [(i, 0.5) for i in range(0, 100)]
202         assert TrackNetRunner._trim_inactive_tail(allfast, 30, 0.3, 0.2) == allfast #
fully-fast: untouched
203         assert TrackNetRunner._trim_inactive_tail(allfast, 0, 0.3, 0.2) == allfast #
trail=0: OFF/no-op
204
205
206     def test_window_inactivity_end_trims_through_api():
207         # one window of fast motion then slow drift (both visible & > velocity_thresh ? one
group).
208         fast = [TrajectoryPoint(frame=i, visible=True, x=100 + 40 * (i % 5), y=300) for i in
range(0, 90)]
209         slow = [TrajectoryPoint(frame=i, visible=True, x=100 + (i % 2), y=300) for i in
range(90, 300)]
210         pts = fast + slow
211         off = TrackNetRunner.trajectory_to_action_windows(
212             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3)
213         on = TrackNetRunner.trajectory_to_action_windows(
214             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3,
215             window_inactivity_end=1.0, inactivity_rally_thresh=0.2,
inactivity_min_density=0.3)
216         assert on and off
217         assert on[0]["end"] < off[0]["end"]                # R4 trims the drift tail ? earlier END
218         assert on[0]["start"] == off[0]["start"]           # START unchanged
219
220
221     # ---- R5: blob-fallback (force-split the continuously-active blob, default OFF)
----- #
222     def test_blob_fallback_chop_pure():
223         blob = [(i, 1.0) for i in range(0, 200)]           # 199-frame span, no gap = 6.6× a
30-frame cap
224         out, flags = TrackNetRunner._blob_fallback_chop([blob], max_win_frames=30,
blob_factor=4.0)
225         assert len(out) >= 4 and all(flags)                # force-chopped, every
piece flagged
226         assert all((g[-1][0] - g[0][0]) <= 30 for g in out) # every piece within
the cap
227         assert sorted(f for g in out for f, _ in g) == list(range(0, 200)) # partition
(only cuts)
228
229
230     def test_blob_fallback_factor_leaves_normal_long_rally_alone():
231         # 79-frame span = 2.6× a 30-frame cap ? BELOW the 4× blob threshold ? left ALONE

```

```

(the gate that
232     # keeps R5 surgical; this is the over-cap-but-not-a-blob case hard_cap would wrongly
chop).
233     normal = [(i, 1.0) for i in range(0, 80)]
234     out, flags = TrackNetRunner._blob_fallback_chop([normal], max_win_frames=30,
blob_factor=4.0)
235     assert out == [normal] and flags == [False]
236
237
238     def test_window_blob_fallback_rescues_blob_through_api():
239         # A continuous ~10s blob (no gap) the cap alone leaves whole: blob_fallback breaks
it up + flags.
240         pts = _moving(0, 301)                                     # 10s continuous, no
internal gap
241         off = TrackNetRunner.trajectory_to_action_windows(
242             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
243             min_window_duration=0.3, max_window_duration=2.0)      # cap 2s, blob_fallback
OFF
244         on = TrackNetRunner.trajectory_to_action_windows(
245             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
246             min_window_duration=0.3, max_window_duration=2.0,
247             window_blob_fallback=True, window_blob_factor=4.0)    # 10s > 4x2s threshold ?
a blob
248         assert len(off) == 1 and (off[0]["end"] - off[0]["start"]) > 9.0    # cap alone
leaves it whole
249         assert len(on) >= 4                                                # the blob is broken up
250         assert all((w["end"] - w["start"]) <= 2.01 for w in on)           # every piece within the
cap
251         assert all(w.get("forced_cut") for w in on)                        # every piece flagged a
forced cut
252
253
254     def test_window_blob_fallback_noop_without_cap():
255         # blob_fallback requires a cap to define "over-cap"; with max_window_duration=0 it's
a no-op.
256         w = TrackNetRunner.trajectory_to_action_windows(
257             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
merge_gap=2.0,
258             min_window_duration=0.3, window_blob_fallback=True)
259         assert len(w) == 1 and not w[0].get("forced_cut")
260
261
262     # ----- WSL subprocess glue
263
264     def _runner():
265         return TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5"))
266
267
268     def test_healthcheck_ok():
269         r = _runner()
270         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF 2.17.0 GPUs 1")):
271             ok, msg = r.healthcheck()
272             assert ok is True
273             assert "TF 2.17.0" in msg
274
275
276     def test_healthcheck_failure_returns_message():
277         r = _runner()
278         with patch.object(r, "_wsl_bash", return_value=_proc(1,
stderr="ModuleNotFoundError")):
279             ok, msg = r.healthcheck()
280             assert ok is False
281             assert "ModuleNotFoundError" in msg
282
283

```

```

284 def test_healthcheck_no_wsl_binary():
285     r = _runner()
286     with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
287         ok, msg = r.healthcheck()
288         assert ok is False
289         assert "wsl" in msg.lower()
290
291
292 def test_run_predict_requires_weights():
293     r = TrackNetRunner(TrackNetConfig(weights_path="")) # no weights configured
294     assert r.run_predict(r"C:\v\clip.mp4", "out") is None
295
296
297 def test_run_predict_rejects_bad_extension(tmp_path):
298     r = _runner()
299     assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is None
300
301
302 def test_run_predict_success(tmp_path):
303     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
304     out_dir = str(tmp_path)
305     # predict.py would write "<stem>_predict.csv"; create it so the existence check
306     csv_path = os.path.join(out_dir, "clip_predict.csv")
307     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="Prediction
complete.")):
308         open(csv_path, "w").close()
309         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
310         assert result == csv_path
311
312
313 def test_run_predict_failure_returns_none(tmp_path):
314     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
315     with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="CUDA error")):
316         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
317
318
319 def test_run_predict_missing_output_csv(tmp_path):
320     """Exit 0 but no CSV produced should be treated as failure."""
321     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
322     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
323         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
324
325
326 def test_stage_video_success():
327     r = _runner()
328     # Two WSL calls now: the readability preflight (echo READABLE), then the cp (echo
329     OK).
330     with patch.object(r, "_wsl_bash",
331                       side_effect=[_proc(0, stdout="READABLE"), _proc(0, stdout="OK"))]:
332         staged = r._stage_video(r"C:\v\clip.mp4", "clip.mp4")
333         assert staged == "~/clips/clip.mp4"
334
335
336 def test_stage_video_failure():
337     r = _runner()
338     # Source readable, but the cp itself fails -> None.
339     with patch.object(r, "_wsl_bash",
340                       side_effect=[_proc(0, stdout="READABLE"), _proc(1, stderr="disk
full"))]:
341         assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None
342
343
344 def test_stage_video_unreadable_source_fails_fast_with_actionable_msg(caplog):
345     """A source invisible to WSL (Google Drive / UNC) must fail BEFORE cp, with a clear,

```

```

345     fixable message ? not a cryptic 'cannot stat' that silently yields 0 rallies.""
346     r = _runner()
347     import logging
348     # Preflight `test -r` returns no READABLE -> unreadable. cp must NOT be attempted.
349     with patch.object(r, "_wsl_bash", return_value=_proc(1, stdout="")) as wsl, \
350         caplog.at_level(logging.ERROR):
351         assert r._stage_video("G:\\My Drive\\Sharing\\clip.mp4", "clip.mp4") is None
352     assert wsl.call_count == 1 # only the preflight ran; cp short-circuited
353     assert "not readable from WSL" in caplog.text
354     assert "local disk" in caplog.text
355
356
357     def test_wsl_source_unreadable_reason_none_when_readable():
358         r = _runner()
359         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="READABLE")):
360             assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
361
362
363     def test_wsl_source_unreadable_reason_none_on_flaky_probe():
364         """"A timed-out/raising probe must NOT block a genuinely-readable source.""
365         r = _runner()
366         with patch.object(r, "_wsl_bash", side_effect=subprocess.TimeoutExpired(cmd="x",
367             timeout=1)):
368             assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
369
370     def test_run_predict_stages_when_enabled(tmp_path):
371         """"With stage_in_wsl=True, run_predict should stage the video first.""
372         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=True))
373         out_dir = str(tmp_path)
374         csv_path = os.path.join(out_dir, "clip_predict.csv")
375         with patch.object(r, "_stage_video", return_value="~/clips/clip.mp4") as mock_stage, \
376             patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
377             open(csv_path, "w").close()
378             result = r.run_predict(r"C:\v\clip.mp4", out_dir)
379         mock_stage.assert_called_once()
380         assert result == csv_path
381
382
383     # ----- DetectorRunner ABC
384     contract
385
386     def test_tracknet_runner_implements_detector_runner_abc():
387         """"TrackNetRunner must be a DetectorRunner (for polymorphism / future native
388         runners).""
389         r = _runner()
390         assert isinstance(r, DetectorRunner)
391         # healthcheck contract: (bool, str) ? exercised with patch in other tests, but shape
392         here too
393         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF ok")):
394             res = r.healthcheck()
395             assert isinstance(res, tuple) and len(res) == 2
396             assert isinstance(res[0], bool)
397             assert isinstance(res[1], str)
398
399     def test_wasb_runner_implements_detector_runner_abc():
400         """"WasbRunner must also satisfy the DetectorRunner contract.""
401         w = WasbRunner(WasbConfig())
402         assert isinstance(w, DetectorRunner)
403         with patch.object(w, "_wsl_bash", return_value=_proc(0, stdout="torch ok")):
404             res = w.healthcheck()
405             assert isinstance(res, tuple) and len(res) == 2
406             assert isinstance(res[0], bool) and isinstance(res[1], str)

```

```

405
406
407 # ----- WASB streaming fast-path wiring
408
409 def test_wasb_config_stream_video_default_off_and_parsing():
410     """Streaming is OFF by default (production behavior unchanged) and opt-in via
411     cfg."""
412     assert WasbConfig().stream_video is False
413     cfg = WasbConfig.from_indexing_cfg({"wasb": {"stream_video": True}})
414     assert cfg.stream_video is True
415     assert WasbConfig.from_indexing_cfg({"wasb": {}}).stream_video is False
416
417 def _wasb_run_predict_cmd(stream_video, tmp_path):
418     """Drive WasbRunner.run_predict with WSL fully mocked and return the inference cmd
419     string (the last _wsl_bash call carries `python wasb_infer.py ...`)."""
420     r = WasbRunner(WasbConfig(stream_video=stream_video))
421     out_dir = str(tmp_path)
422     key = "clip__abc123abc123"
423     csv_path = os.path.join(out_dir, f"{key}_wasb.csv")
424     with patch.object(r, "_stage_infer_script", return_value=True), \
425         patch.object(r, "_stage_video", return_value="~/clips/clip.mp4"), \
426         patch.object(r, "_wsl_rm_rf"), \
427         patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")) as wsl:
428         open(csv_path, "w").close()
429         result = r.run_predict(r"C:\v\clip.mp4", out_dir, video_id="abc123abc123def")
430     assert result == csv_path
431     # The inference invocation is the call whose command contains wasb_infer.py.
432     infer_cmds = [c.args[0] for c in wsl.call_args_list if "wasb_infer.py" in c.args[0]]
433     assert len(infer_cmds) == 1
434     return infer_cmds[0]
435
436
437 def test_run_predict_streaming_passes_stream_flag_not_frames_dir(tmp_path):
438     cmd = _wasb_run_predict_cmd(stream_video=True, tmp_path=tmp_path)
439     assert "--stream-video" in cmd
440     assert "--frames_out_dir" not in cmd # no PNG extraction on the fast path
441     assert "--video " in cmd # still decodes the staged video
442
443
444 def test_run_predict_default_passes_frames_dir_not_stream_flag(tmp_path):
445     cmd = _wasb_run_predict_cmd(stream_video=False, tmp_path=tmp_path)
446     assert "--stream-video" not in cmd # default path unchanged
447     assert "--frames_out_dir" in cmd
448
449
450 # (Intentionally no "no-wsl unpatched predict" test here: the stage/inference paths
451 # raise FileNotFoundError on this Linux box when "wsl" is absent, while only the
452 # healthcheck paths catch it today. The graceful-None cases are already exercised
453 # by the patched tests above and by the hybrid tests. Adding a broad except in
454 # run_predict would be a separate robustness change.)
455

```

25. assistant (2026-06-18T07:55:16.273Z)

Excellent ? `trajectory\_to\_action\_windows` and all its R4/R5/W1 helpers have very strong, direct unit-test coverage. Now let me read the compiler test and check the `windowing.py` eval module (which re-implements some of this) to assess dedup opportunities.

26. user (2026-06-18T07:55:17.390Z)

```

1     """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2     where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3     import os
4     import subprocess

```

```

5     from unittest.mock import patch, MagicMock
6
7     import pytest
8
9     from backend.pipeline.stitcher.compiler import VideoCompiler
10
11
12     def test_parse_time_formats():
13         f = VideoCompiler._parse_time
14         assert f(12.5) == 12.5
15         assert f("12.5") == 12.5
16         assert f("1:30") == 90.0
17         assert f("1:00:00") == 3600.0
18         assert f("garbage") == 0.0
19
20
21     def test_merge_overlapping_collapses_duplicates_and_overlaps():
22         m = VideoCompiler._merge_overlapping
23         # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
to one each.
24         assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [(28.2, 34.5),
(40.5, 45.5)]
25         # Unsorted input is sorted; an overlapping range extends the previous.
26         assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [(0.0, 5.0), (100.0,
120.0)]
27         # Genuinely distinct (gapped) rallies are preserved, not merged.
28         assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
29         # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
30         assert m([("0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]
31
32
33     def test_compile_dedups_before_stitching(tmp_path):
34         """A segment list with every rally twice must yield ONE clip per unique rally."""
35         comp = VideoCompiler(str(tmp_path / "out"))
36         raw = tmp_path / "raw.mp4"
37         raw.write_bytes(b"x")
38         trims = []
39
40         def fake_run(cmd, *a, **k):
41             if cmd[0] == "ffprobe":
42                 return MagicMock(stdout="100.0\n", returncode=0)
43             out = cmd[-1]
44             if isinstance(out, str) and out.endswith(".mp4"):
45                 open(out, "wb").write(b"clip")
46                 # count per-clip trims (the concat list file input is not an .mp4 output)
47                 if "-i" in cmd and cmd.index("-i") + 1 == str(raw):
48                     trims.append(cmd)
49             return MagicMock(returncode=0, stdout=b"", stderr=b"")
50
51         dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
52         with patch.object(subprocess, "run", side_effect=fake_run):
53             comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
54             # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
55             assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"
56
57
58     def test_compile_basenames_output_and_stitches(tmp_path):
59         comp = VideoCompiler(str(tmp_path / "out"))
60         raw = tmp_path / "raw.mp4"
61         raw.write_bytes(b"x")
62
63         calls = []
64
65         def fake_run(cmd, *a, **k):
66             calls.append(cmd)

```

```

67         # Create whatever output file ffmpeg/ffprobe was told to write so existence
checks pass.
68         if cmd[0] == "ffprobe":
69             return MagicMock(stdout="100.0\n", returncode=0)
70         out = cmd[-1]
71         if isinstance(out, str) and out.endswith(("mp4",)):
72             open(out, "wb").write(b"clip")
73             return MagicMock(returncode=0, stdout=b"", stderr=b"")
74
75     with patch.object(subprocess, "run", side_effect=fake_run):
76         # Traversal in the name must be stripped (defense-in-depth basename).
77         out_path = comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)],
"../reel.mp4")
78
79         assert os.path.basename(out_path) == "reel.mp4"
80         assert os.path.dirname(out_path).endswith("out")
81         # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
82         assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3
83
84
85     def test_compile_raises_on_no_segments(tmp_path):
86         comp = VideoCompiler(str(tmp_path / "out"))
87         raw = tmp_path / "raw.mp4"
88         raw.write_bytes(b"x")
89         with pytest.raises(ValueError):
90             comp.compile_highlights(str(raw), [], "reel.mp4")
91
92
93     def test_compile_raises_when_source_missing(tmp_path):
94         comp = VideoCompiler(str(tmp_path / "out"))
95         with pytest.raises(FileNotFoundError):
96             comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
97

```

27. assistant (2026-06-18T07:55:20.888Z)

Now let me check the `windowing.py` eval module to see if it duplicates `trajectory\_to\_action\_windows` logic (potential dedup target), and look at the person_detector test for the windowing duplication there too.

28. user (2026-06-18T07:55:21.659Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
?
4     `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5     on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
6     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
merge_gap=2.0,
8     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
is
9     measured on candidates **production never serves**, so a "win" need not transfer. This
is
10    exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
set
11    (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
12
13    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`, and
the
14    SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**

```

```

15     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17     *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18     operating point a given deployment truly serves.
19
20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21     windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22     ``per_user_gate``). This module is the what windowing; that one is the promotion
gate.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset is the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.

```

```

66         hard_cap: bool = False
67         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69         inactivity_end: float = 0.0
70         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71         inactivity_rally_thresh: float = 0.08
72         inactivity_min_density: float = 0.34
73         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76         #: factor keeps it surgical (a normal long rally is left alone).
77         blob_fallback: bool = False
78         blob_factor: float = 4.0
79
80         def as_tuple(self) -> Tuple[float, float, float]:
81             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83             bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86         def to_dict(self) -> Dict[str, Any]:
87             return {
88                 "name": self.name,
89                 "velocity_thresh": self.velocity_thresh,
90                 "merge_gap": self.merge_gap,
91                 "min_window_duration": self.min_window_duration,
92                 "max_window_duration": self.max_window_duration,
93                 "hard_cap": self.hard_cap,
94                 "inactivity_end": self.inactivity_end,
95                 "inactivity_rally_thresh": self.inactivity_rally_thresh,
96                 "inactivity_min_density": self.inactivity_min_density,
97                 "blob_fallback": self.blob_fallback,
98                 "blob_factor": self.blob_factor,
99             }
100
101
102         def _served_preset_from_defaults() -> WindowingPreset:
103             """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105             ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106             - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107             - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108             - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110             Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111             point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112             so the two can never silently diverge again.
113             """
114             idx = IndexingConfig() # construct with defaults ? the served operating point
115             return WindowingPreset(
116                 name="served",
117                 velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet

```

```

default 0.02
118         merge_gap=float(idx.dead_time_merge_gap),
119         min_window_duration=float(idx.min_action_window_duration),
120         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121         hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122         inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",

```

```

SERVED.inactivity_rally_thresh)),
166         inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167         blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168         blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169     )
170
171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
174         (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175         if obj is None:
176             return {}
177         if isinstance(obj, Mapping):
178             sub = obj.get(key, {})
179         elif hasattr(obj, "get"):
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                     preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
189
190         A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211

```

29. assistant (2026-06-18T07:55:25.487Z)

Now I understand the windowing well ? it's already single-sourced. Let me look at the person_detector test and check the duplicated grouping/_finalize` logic between person\_detector and tracknet\_runner (a possible dedup). Let me also check the `wasb_runner.py` to confirm the cluster boundaries.

30. user (2026-06-18T07:55:26.186Z)

```

1      """Unit tests for the person-detection rally cue
(backend.pipeline.detectors.person_detector).
2
3      Pure/offline ? the torchvision model and supervision tracker are mocked, so these run
4      without GPU/torch weights. They pin the COCO person==1 filter (the YOLO-was-0 gotcha)
and
5      the motion-window extraction that feeds the candidate `stats` + the learned fusion
features.
6      """
7      from unittest.mock import MagicMock, patch
8
9      from backend.pipeline.detectors.person_detector import PersonDetector, _PERSON_CLASS_ID
10
11
12      def _make_cap_mock(num_frames=300):
13          """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
14          import cv2
15          cap_mock = MagicMock()
16          cap_mock.isOpened.return_value = True
17
18          def mock_get_prop(prop):
19              if prop == cv2.CAP_PROP_FPS:
20                  return 30.0
21              if prop == cv2.CAP_PROP_FRAME_WIDTH:
22                  return 1000.0
23              return 0.0
24
25          cap_mock.get.side_effect = mock_get_prop
26          cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
27          return cap_mock
28
29
30      def _make_detections(num_frames=300, step=5, track_id=1):
31          """One steadily-moving person per frame (box advances `step` px/frame)."""
32          return [(track_id, (float(i * step), 0.0, float(i * step + 10), 10.0))]
33                  for i in range(num_frames)]
34
35
36      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
37      def test_extract_action_windows_returns_enriched_dicts(mock_cap):
38          pd = PersonDetector({"indexing": {"min_action_window_duration": 1.0}})
39          pd.model = MagicMock() # short-circuits
lazy_load_model
40          pd.make_tracker = MagicMock(return_value=MagicMock())
41          pd.detect_and_track = MagicMock(side_effect=_make_detections())
42          mock_cap.return_value = _make_cap_mock()
43
44          windows = pd.extract_action_windows_from_chunk(
45              "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
46              frame_skip=1, chunk_index=2,
47          )
48
49          assert len(windows) >= 1
50          w = windows[0]
51          assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
52                           "mean_velocity", "track_id_count", "chunk_index"}
53          assert w["chunk_index"] == 2
54          assert w["start"] >= 5.0 # full-video coordinates
55          assert w["active_frame_count"] > 0
56          assert w["peak_velocity"] >= w["mean_velocity"] > 0
57          assert w["track_id_count"] == 1
58
59
60      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
61      def test_detections_per_frame(mock_cap):

```

```

62     """The per-frame path the fusion segmenter uses: samples [start,end] by frame_skip,
63     returns {frame_idx: dets} keyed by absolute video frame + the frame dims (incl.
height)."""
64     import cv2
65
66     pd = PersonDetector({"indexing": {"detector_score_threshold": 0.5}})
67     pd.model = MagicMock() # short-circuits
lazy_load_model
68     pd.make_tracker = MagicMock(return_value=MagicMock())
69     pd.detect_and_track = MagicMock(return_value=[(1, (10.0, 10.0, 30.0, 40.0))])
70
71     cap = _make_cap_mock(num_frames=20)
72
73     def _get(prop):
74         if prop == cv2.CAP_PROP_FPS:
75             return 30.0
76         if prop == cv2.CAP_PROP_FRAME_WIDTH:
77             return 1000.0
78         if prop == cv2.CAP_PROP_FRAME_HEIGHT:
79             return 800.0
80         return 0.0
81
82     cap.get.side_effect = _get
83     mock_cap.return_value = cap
84
85     out = pd.detections_per_frame("v.mp4", start_frame=0, end_frame=9, frame_skip=3)
86     assert out["fps"] == 30.0
87     assert out["frame_width"] == 1000.0 and out["frame_height"] == 800.0 # height
returned (the gap)
88     assert sorted(out["frames"].keys()) == [0, 3, 6, 9] # sampled every 3rd frame in
[0,9]
89     assert out["frames"][0] == [(1, (10.0, 10.0, 30.0, 40.0))]
90     cap.release.assert_called_once()
91
92
93     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
94     def test_detections_over_windows_single_pass(mock_cap):
95         """One sequential pass dispatches sampled frames to the right windows (no per-window
seek)."""
96         import cv2
97
98         pd = PersonDetector({"indexing": {"detector_score_threshold": 0.5}})
99         pd.model = MagicMock()
100         pd.make_tracker = MagicMock(return_value=MagicMock())
101         pd.detect_and_track = MagicMock(return_value=[(1, (10.0, 10.0, 30.0, 40.0))])
102
103         cap = MagicMock()
104         cap.isOpened.return_value = True
105         cap.get.side_effect = lambda p: {cv2.CAP_PROP_FPS: 30.0, cv2.CAP_PROP_FRAME_WIDTH:
1000.0,
106                                         cv2.CAP_PROP_FRAME_HEIGHT: 800.0}.get(p, 0.0)
107         cap.grab.side_effect = [True] * 11 + [False] # frames 0..10 readable
108         cap.retrieve.return_value = (True, "frame")
109         mock_cap.return_value = cap
110
111         windows = [(0, 6), (8, 10)] # frame_skip=2
112         out = pd.detections_over_windows("v.mp4", windows, frame_skip=2)
113
114         assert len(out) == 2
115         assert out[0]["frame_width"] == 1000.0 and out[0]["frame_height"] == 800.0
116         assert sorted(out[0]["frames"].keys()) == [0, 2, 4, 6] # window 0 on its sampling
grid
117         assert sorted(out[1]["frames"].keys()) == [8, 10] # window 1
118         mock_cap.assert_called_once() # opened ONCE for all
windows

```

```

119
120
121 def test_detect_and_track_filters_persons_and_returns_track_ids():
122     """detect_and_track keeps only confident PERSON detections, feeds them to the
123     tracker, and returns (track_id, bbox) tuples ? exercising the real torchvision
124     output-shape handling and the COCO person==1 filter (the YOLO-was-0 gotcha)."""
125     import numpy as np
126
127     pd = PersonDetector({"indexing": {}})
128     pd.device = "cpu"
129
130     # Fake torchvision detector output: a person (kept), a low-score person (dropped),
131     # and a racket/other class (dropped).
132     out = {"boxes": MagicMock(), "labels": MagicMock(), "scores": MagicMock()}
133     out["boxes"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
134         [[0, 0, 10, 10], [5, 5, 15, 15], [20, 20, 30, 30]], dtype=float)
135     out["labels"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
136         [_PERSON_CLASS_ID, _PERSON_CLASS_ID, 38]) # 38 = sports-ball-ish, not
person
137     out["scores"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
138         [0.9, 0.1, 0.99])
139     pd.model = MagicMock(return_value=out)
140
141     # Fake tracker echoes the detections it receives, assigning a single track id.
142     tracker = MagicMock()
143     tracked = MagicMock()
144     tracked.__len__.return_value = 1
145     tracked.tracker_id = [7]
146     tracked.xyxy = np.array([[0, 0, 10, 10]], dtype=float)
147     tracker.update_with_detections.return_value = tracked
148
149     with patch("backend.pipeline.detectors.person_detector.cv2.cvtColor",
return_value=np.zeros((10, 10, 3))), \
150         patch("backend.pipeline.detectors.person_detector.torch") as mock_torch, \
151         patch.dict("sys.modules", {"supervision": MagicMock()}):
152         mock_torch.no_grad.return_value.__enter__ = lambda s: None
153         mock_torch.no_grad.return_value.__exit__ = lambda s, *a: None
154         # torch.from_numpy(...).permute(...).float().div(...).to(...) -> dummy tensor
155         mock_torch.from_numpy.return_value.permute.return_value.float.return_value.div.r
eturn_value.to.return_value = "tensor"
156         result = pd.detect_and_track("frame", tracker, score_thresh=0.5)
157
158     # Only the high-confidence person survived the score+class filter
159     assert result == [(7, (0.0, 0.0, 10.0, 10.0))]
160     # And exactly that filtered set (1 box) was handed to the tracker
161     tracker.update_with_detections.assert_called_once()
162

```

31. assistant (2026-06-18T07:55:30.473Z)

Now I have a clear picture of test coverage. Let me look at the duplicated grouping logic between person_detector and tracknet_runner, and confirm whether the `run()` function in wasb_infer has any extractable seam with test protection. Let me check git history for how recently these files changed (to gauge churn/blame risk) and run the existing tests to confirm green baseline.

32. user (2026-06-18T07:55:40.749Z)

```

..... [ 87%]
..... [100%]
82 passed in 6.22s

```

33. assistant (2026-06-18T07:55:45.778Z)

Green baseline. Now let me examine the structural duplication between the two `\_finalize`/grouping blocks, and check the `run()` function's internal seams for a safe extraction. Let me look at the imports in person_detector vs tracknet to see if a shared helper is feasible without behavior change.

The two `\_finalize` functions differ (person tracks `track\_id\_count` via set union; tracknet hardcodes 1), and grouping is inline-different. Let me verify there's no existing shared windowing util and check `geometry.py`.

34. user (2026-06-18T07:55:47.317Z)

```
backend\main.py:105:def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int)
-> None:
backend\utils\geometry.py:23:def get_velocity_normalised(bbox1: Tuple[float, float, float,
float], bbox2: Tuple[float, float, float, float],
backend\pipeline\detectors\person_detector.py:227:         max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\person_detector.py:229:         def _finalize(group: List[Tuple[int,
float, set]]) -> Dict[str, Any]:
backend\pipeline\detectors\person_detector.py:234:             "end": group[-1][0] / fps +
chunk_start,
backend\pipeline\detectors\person_detector.py:245:             if af[0] - group[-1][0] <=
max_gap_frames:
backend\pipeline\detectors\tracknet_runner.py:334:         max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\tracknet_runner.py:336:         def _finalize(group: List[Tuple[int,
float]]) -> Dict[str, Any]:
backend\pipeline\detectors\tracknet_runner.py:340:             "end": group[-1][0] / fps +
chunk_start,
backend\pipeline\detectors\tracknet_runner.py:351:             if af[0] - group[-1][0] <=
max_gap_frames:
```

35. assistant (2026-06-18T07:55:53.493Z)

The grouping is structurally similar but the tuple shapes and `\_finalize` payloads genuinely differ (3-tuple with id-set vs 2-tuple) ? a dedup here would be a redesign with subtle behavior risk, not a pure move. I'll note that but not propose it as a safe refactor.

Now let me assess the `wasb\_infer.run()` function ? the 185-line one. The detector-pass loop body (lines 471-528) is a coherent block that could extract, but it's wired to torch/DataLoader and has NO test coverage (the test file covers only pure helpers). Let me confirm there are no other callers/tests touching `run()` directly.

36. user (2026-06-18T07:55:55.286Z)

```
backend\pipeline\detectors\wasb_infer.py:630:def run_video_streaming(video_path: str, weights:
str, out_csv: str, sport: str = "badminton",
backend\pipeline\detectors\wasb_infer.py:681:         run_video_streaming(args.video,
args.weights, args.out, args.sport, args.limit,
backend\pipeline\segmenters\gemini.py:11:from backend.reporting import run_signals
backend\reporting\__init__.py:12:from . import run_signals # no obsreport dependency ? always
importable
tests\test_eval_harness.py:11:import run_eval
tests\test_eval_regression.py:7:import run_regression
tests\test_fusion_golden_telemetry.py:14:from backend.utils import run_telemetry as rt
tests\test_gemini.py:6:from backend.reporting import run_signals
tests\test_import_smoke.py:9:anything, and the backend.main import runs in a subprocess with any
missing
tests\test_run_telemetry.py:9:from backend.utils import run_telemetry as rt
```

37. assistant (2026-06-18T07:56:01.319Z)

`wasb\_infer.run()` is invoked only from within the same module (via `run\_video\_streaming` and `main`), and it's never imported/tested directly ? it runs on the WSL GPU box. So any extraction inside `run()` has NO automated test protection. That makes `run()` extractions higher-risk; I'll flag them as "needs tests first" rather than propose them as safe.

Now let me look at one promising low-risk, well-tested seam: in `tracknet\_runner.trajectory\_to\_action\_windows`, the velocity-computation + grouping is inline but the post-processing (split/trim/blob) is already extracted to tested static methods. Let me check whether there are any genuinely safe targets. Let me re-read the velocity/grouping block boundaries and confirm the `geometry.py` helper.

38. user (2026-06-18T07:56:02.073Z)

```
1     import math
2     from typing import Tuple
3
4     def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5         """Calculate the center (x, y) of a bounding box.
6         bbox format: (x1, y1, x2, y2)
7         """
8         x1, y1, x2, y2 = bbox
9         return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11    def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12        """Calculate the Euclidean distance between two points (x, y)."""
13        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15    def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
16        """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
17        """
18        c1 = get_center(bbox1)
19        c2 = get_center(bbox2)
20        dist = get_distance(c1, c2)
21        return dist * fps
22
23    def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
24                                fps: float, frame_width: float) -> float:
25        """Calculate velocity as a fraction of frame width per second.
26
27        This makes the threshold resolution-independent:
28        - 0.15 means the center moved 15% of the frame width in one second.
29        - Works identically on 720p, 1080p, and 4K footage.
30
31        Args:
32            bbox1: Previous bounding box (x1, y1, x2, y2).
33            bbox2: Current bounding box (x1, y1, x2, y2).
34            fps: Effective frames per second (after frame-skipping).
35            frame_width: Width of the video frame in pixels.
36        """
37        raw_velocity = get_velocity(bbox1, bbox2, fps)
38        if frame_width <= 0:
39            return raw_velocity # Fallback to raw pixels if width unknown
40        return raw_velocity / frame_width
41
42    def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
43        """Calculate the Intersection over Union (IoU) of two bounding boxes.
44        bbox format: (x1, y1, x2, y2)
45        """
46        xA = max(boxA[0], boxB[0])
47        yA = max(boxA[1], boxB[1])
48        xB = min(boxA[2], boxB[2])
49        yB = min(boxA[3], boxB[3])
50
51        interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
52        if interArea == 0:
53            return 0.0
```

```

54
55         boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
56         boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
57
58         iou = interArea / float(boxAArea + boxBArea - interArea)
59         return iou
60

```

39. assistant (2026-06-18T07:56:11.182Z)

Now I have a comprehensive picture. Let me identify the genuinely safe, well-tested seams within the cluster.

Looking at `compiler.compile\_highlights` (202L) ? the per-segment trim loop (lines 234-335, ~100 lines) is a clear extract-method seam. It's a coherent unit: validate one segment ? compute padded bounds ? ffmpeg-trim with watermark-retry ? return clip path or None. The compiler test mocks `subprocess.run` and asserts on trim counts, dedup, basename, and error paths ? so an extraction that preserves the exact ffmpeg command sequence and skip/clamp behavior would be caught if it regressed. Let me verify the test coverage depth on the clamping/skipping branches.

The compiler test covers: dedup (3 trims), basename stripping, no-segments raise, missing-source raise, and stitch invocation count. It does NOT cover the watermark-retry path, negative-start clamping, out-of-range skipping, or empty-output handling. So coverage is **partial** ? the happy path and dedup are well-tested, but several branches inside the trim loop are not. An extraction of the whole loop body is safe for the structure but a logic slip in an untested branch (e.g. clamp ordering) wouldn't be caught.

Let me check one more thing ? whether `compile\_highlights` is called elsewhere and whether there's a watermark test anywhere.

40. user (2026-06-18T07:56:12.903Z)

```

Found 4 files
tests\test_watermark.py
tests\test_generate_highlights.py
tests\test_compiler.py
tests\test_api_endpoints.py

```

41. user (2026-06-18T07:56:15.652Z)

```

1      """Tests for the configurable highlight watermark (stitching.watermark).
2
3      Covers the config defaults, the filtergraph builder (text + logo, positions, font,
4      the configurable string), and the end-to-end wiring through compile_highlights with
5      ffmpeg/ffprobe mocked ? including the graceful fallback when the watermark encode fails.
6      """
7      import os
8      import subprocess
9      from pathlib import Path
10     from unittest.mock import MagicMock, patch
11
12     import pytest
13     from pydantic import ValidationError
14
15     from backend.config import AppConfig
16     from backend.config.models import WatermarkConfig
17     from backend.pipeline.stitcher.compiler import VideoCompiler
18
19
20     # --- config -----
21
22     def test_watermark_config_defaults_on_with_brand_text():
23         wm = AppConfig().stitching.watermark
24         assert wm.enabled is True # default-on, with an off switch

```

```

25         assert wm.text == "Generated by Khelsutra.guru"
26         assert wm.position == "bottom-right"
27
28
29     def test_watermark_can_be_disabled_via_config():
30         cfg = AppConfig.model_validate({"stitching": {"watermark": {"enabled": False}}})
31         assert cfg.stitching.watermark.enabled is False
32
33
34     def test_font_size_ratio_and_opacity_bounds_enforced():
35         with pytest.raises(ValidationError):
36             WatermarkConfig(opacity=1.5)
37         with pytest.raises(ValidationError):
38             WatermarkConfig(font_size_ratio=0.0)
39
40
41     # --- filtergraph builder -----
42
43     def test_disabled_or_none_builds_no_filter(tmp_path):
44         assert VideoCompiler(str(tmp_path / "a"),
watermark=None)._watermark_filter(str(tmp_path)) is None
45         off = VideoCompiler(str(tmp_path / "b"), watermark={"enabled": False})
46         assert off._watermark_filter(str(tmp_path)) is None
47
48
49     def test_text_filter_writes_configurable_string_and_builds_drawtext(tmp_path):
50         custom = "My Academy 2026 ? Khelsutra.guru"
51         comp = VideoCompiler(str(tmp_path / "o"), watermark={"enabled": True, "text":
custom})
52         graph = comp._watermark_filter(str(tmp_path))
53         assert graph is not None and graph.startswith("drawtext=")
54         # The arbitrary, configurable string is written verbatim to the textfile (no
escaping pain).
55         assert (tmp_path / "watermark.txt").read_text(encoding="utf-8") == custom
56         assert "textfile=" in graph
57         assert "fontcolor=white@0.85" in graph
58         assert "fontsize=h/29" in graph                # round(1/0.035) == 29
59         assert "x=w-tw-24:y=h-th-24" in graph           # bottom-right default, margin 24
60         assert "box=1" in graph
61
62
63     def test_position_and_margin_map_to_corner(tmp_path):
64         comp = VideoCompiler(str(tmp_path / "o"),
65                             watermark={"enabled": True, "text": "X", "position": "top-
left", "margin": 12})
66         graph = comp._watermark_filter(str(tmp_path))
67         assert "x=12:y=12" in graph
68
69
70     def test_explicit_font_path_uses_fontfile(tmp_path):
71         comp = VideoCompiler(str(tmp_path / "o1"),
72                             watermark={"enabled": True, "text": "X", "font_path":
"/fonts/Brand.ttf"})
73         g = comp._watermark_filter(str(tmp_path))
74         assert "fontfile='/fonts/Brand.ttf'" in g
75
76
77     def test_default_uses_bundled_apache_font(tmp_path):
78         # No font_path ? the bundled Apache-2.0 Roboto Slab (renders without fontconfig).
79         comp = VideoCompiler(str(tmp_path / "o2"), watermark={"enabled": True, "text": "X"})
80         g = comp._watermark_filter(str(tmp_path))
81         assert "fontfile=" in g and "RobotoSlab-Regular.ttf" in g
82         assert "font='" not in g # not the fontconfig path
83
84

```

```

85     def test_bundled_font_actually_ships_and_is_apache():
86         import backend.pipeline.stitcher.compiler as cmod
87         assert cmod._BUNDLED_FONT.exists(), "bundled default watermark font must ship in the
repo"
88         with open(cmod._BUNDLED_FONT, "rb") as fh:
89             assert fh.read(4) in (b"\x00\x01\x00\x00", b"OTTO", b"true", b"ttcf") # valid
sfnt
90         lic = cmod._BUNDLED_FONT.parent / "LICENSE-Apache-2.0.txt"
91         assert lic.exists() and "Apache License" in lic.read_text(encoding="utf-8",
errors="replace")
92
93
94     def test_fontconfig_fallback_when_bundled_missing(tmp_path, monkeypatch):
95         import backend.pipeline.stitcher.compiler as cmod
96         monkeypatch.setattr(cmod, "_BUNDLED_FONT", Path("/no/such/font.ttf"))
97         comp = VideoCompiler(str(tmp_path / "o3"),
98                             watermark={"enabled": True, "text": "X", "font": "DejaVu
Sans"})
99         g = comp._watermark_filter(str(tmp_path))
100        assert "font='DejaVu Sans'" in g and "fontfile=" not in g
101
102
103    # --- Windows drive-letter colon escaping (regression) -----
104    # A Windows absolute path's drive-letter colon (C:/...) must be escaped for the
105    # ffmpeg filtergraph, or drawtext/movie parse ':' as an option separator and the
106    # encode fails with "Invalid argument" ? which silently disabled the on-by-default
107    # bundled-font watermark (and absolute logo paths) on Windows. Verified end-to-end
108    # against real ffmpeg before this fix landed.
109
110    def test_ff_quote_escapes_drive_letter_colon():
111        assert VideoCompiler._ff_quote(r"C:\fonts\Brand.ttf") == r"C\:/fonts/Brand.ttf"
112        # no colon -> unchanged (POSIX paths and the Linux bundled font are unaffected)
113        assert VideoCompiler._ff_quote("/usr/share/fonts/font.ttf") ==
"/usr/share/fonts/font.ttf"
114        # colon + single-quote + backslash all handled
115        assert VideoCompiler._ff_quote(r"D:\a\b\f.ttf") == r"D\:/a\b/f.ttf"
116
117
118    def test_watermark_font_path_with_drive_colon_is_escaped(tmp_path):
119        # Explicit font_path (literal, existence not required) -> deterministic on any OS.
120        comp = VideoCompiler(str(tmp_path / "o"),
121                            watermark={"enabled": True, "text": "X", "font_path":
"C:/fonts/Brand.ttf"})
122        g = comp._watermark_filter(str(tmp_path))
123        assert r"fontfile='C\:/fonts/Brand.ttf'" in g
124        assert "fontfile='C:/fonts" not in g # the raw colon would break drawtext parsing
125
126
127    def test_logo_plus_text_overlays_then_draws(tmp_path):
128        logo = tmp_path / "logo.png"
129        logo.write_bytes(b"\x89PNG\r\n")
130        comp = VideoCompiler(str(tmp_path / "o"),
131                            watermark={"enabled": True, "text": "X", "logo_path":
str(logo)})
132        graph = comp._watermark_filter(str(tmp_path))
133        assert graph.startswith("movie=")
134        assert "overlay=W-w-24:H-h-24" in graph
135        assert "drawtext=" in graph
136
137
138    def test_missing_logo_is_skipped_gracefully(tmp_path):
139        comp = VideoCompiler(str(tmp_path / "o"),
140                            watermark={"enabled": True, "text": "X", "logo_path":
str(tmp_path / "nope.png")})
141        graph = comp._watermark_filter(str(tmp_path))

```

```

142         assert "movie=" not in graph and graph.startswith("drawtext=") # text still drawn
143
144
145     # --- end-to-end wiring (ffmpeg/ffprobe mocked) -----
146
147     def _fake_run_factory(calls, fail_on_vf=False):
148         def fake_run(cmd, *a, **k):
149             calls.append(cmd)
150             if cmd[0] == "ffprobe":
151                 return MagicMock(stdout="100.0\n", returncode=0)
152             if fail_on_vf and "-vf" in cmd:
153                 raise subprocess.CalledProcessError(1, cmd, output=b"", stderr=b"font not
found")
154             out = cmd[-1]
155             if isinstance(out, str) and out.endswith(".mp4"):
156                 open(out, "wb").write(b"clip")
157             return MagicMock(returncode=0, stdout=b"", stderr=b"")
158         return fake_run
159
160
161     def test_compile_applies_watermark_vf_to_each_clip_not_concat(tmp_path):
162         raw = tmp_path / "raw.mp4"
163         raw.write_bytes(b"x")
164         comp = VideoCompiler(str(tmp_path / "out"),
165                               watermark={"enabled": True, "text": "Generated by
Khelsutra.guru"})
166         calls = []
167         with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls)):
168             comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
169
170         trim_cmds = [c for c in calls if c[0] == "ffmpeg" and "-i" in c and "concat" not in
c]
171         assert trim_cmds, "expected per-clip trim invocations"
172         for c in trim_cmds:
173             assert "-vf" in c
174             assert "drawtext=" in c[c.index("-vf") + 1]
175         # The concat (-c copy) must NOT re-filter ? the mark is already baked into each
clip.
176         concat_cmds = [c for c in calls if "concat" in c]
177         assert concat_cmds and all("-vf" not in c for c in concat_cmds)
178
179
180     def test_compile_without_watermark_adds_no_vf(tmp_path):
181         raw = tmp_path / "raw.mp4"
182         raw.write_bytes(b"x")
183         comp = VideoCompiler(str(tmp_path / "out")) # no watermark passed
184         calls = []
185         with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls)):
186             comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
187         assert all("-vf" not in c for c in calls)
188
189
190     def test_watermark_failure_falls_back_to_unwatermarked_clip(tmp_path):
191         """A bad font/logo must not nuke the reel: the clip re-encodes without the
watermark."""
192         raw = tmp_path / "raw.mp4"
193         raw.write_bytes(b"x")
194         comp = VideoCompiler(str(tmp_path / "out"),
195                               watermark={"enabled": True, "text": "Generated by
Khelsutra.guru"})
196         calls = []
197         with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls,
fail_on_vf=True)):
198             out_path = comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
199

```

```

200     # Reel still produced despite the watermark encode failing.
201     assert out_path.endswith("reel.mp4")
202     assert os.path.exists(out_path)
203     # For the clip, both a watermarked attempt and a plain fallback attempt were made.
204     clip_cmds = [c for c in calls if c[0] == "ffmpeg" and "concat" not in c and "-i" in
205 c]
206     assert any("-vf" in c for c in clip_cmds)          # watermark attempt
207     assert any("-vf" not in c for c in clip_cmds)      # fallback attempt
208
209     def test_watermark_failure_logs_the_ffmpeg_reason(tmp_path, caplog):
210         """The fallback warning must surface ffmpeg's stderr (the *why*), not go silent ?
211         e.g. the 'Fontconfig error' that tells the user to set font_path."""
212         import logging as _logging
213         raw = tmp_path / "raw.mp4"
214         raw.write_bytes(b"x")
215         comp = VideoCompiler(str(tmp_path / "out"), watermark={"enabled": True, "text":
216 "Y"})
217
218         def fake_run(cmd, *a, **k):
219             if cmd[0] == "ffprobe":
220                 return MagicMock(stdout="100.0\n", returncode=0)
221             if "-vf" in cmd:
222                 raise subprocess.CalledProcessError(
223                     1, cmd, output=b"", stderr=b"Fontconfig error: Cannot load default
224 config file")
225             out = cmd[-1]
226             if isinstance(out, str) and out.endswith(".mp4"):
227                 open(out, "wb").write(b"clip")
228             return MagicMock(returncode=0, stdout=b"", stderr=b"")
229
230         with caplog.at_level(_logging.WARNING):
231             with patch.object(subprocess, "run", side_effect=fake_run):
232                 comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
233         assert "Fontconfig error" in caplog.text

```

42. assistant (2026-06-18T07:56:24.133Z)

This significantly changes my assessment of the compiler. Between `test\_compiler.py` and `test\_watermark.py`, `compile\_highlights` has **strong** end-to-end coverage: the per-clip trim loop, the watermark `-vf` application, the watermark-failure fallback (both attempts asserted), the concat `-c copy` invariant, dedup, basename stripping, and the empty/missing-source raises are all exercised with mocked ffmpeg asserting on the exact command sequences. The only branches not directly covered are negative-start clamping and out-of-range segment skipping ? but the dominant trim-loop structure is well-protected.

That makes the **compiler trim-loop extraction** the strongest candidate: high value (202L function, the longest in the cluster), low risk (the extracted method's ffmpeg command sequence is asserted by `test\_watermark.py`'s `trim\_cmds` checks), and cleanly separable.

I now have enough to assess all candidates. Let me do a final check on `wsl\_runner` to confirm whether `\_split\_overlong\_groups`/`\_blob\_fallback\_chop`/`\_trim\_inactive\_tail` share a midpoint-chop helper worth deduping (they each re-implement the iterative worklist midpoint chop).

43. user (2026-06-18T07:56:26.198Z)

```

448:         mid = (g[0][0] + g[-1][0]) / 2.0
449:         split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
493:         mid = (h[0][0] + h[-1][0]) / 2.0
494:         split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) # noqa:
B023

```

44. assistant (2026-06-18T07:56:34.498Z)

Confirmed: there's a small, exact duplication ? the "find the split index nearest the temporal midpoint" two-liner appears in both `\_split\_overlong\_groups` (hard_cap branch) and `\_blob\_fallback\_chop`. Both are covered by direct pure unit tests (`test\_split\_overlong\_groups\_hard\_cap\_pure`, `test\_blob\_fallback\_chop\_pure`). This is a tiny, well-tested dedup candidate.

I now have a complete, evidence-based picture. Let me compile the proposals.

Summary of findings:

- **wasb_infer.py** pure helpers: very strong unit coverage (`test\_wasb\_cache.py`). The `run()` GPU orchestration (185L) has **zero** automated coverage and is WSL-only ? extractions there are unsafe without tests first.
- **tracknet_runner.trajectory_to_action_windows** (231L) and all its W1/R4/R5 helpers: **strong, direct** unit coverage. The post-processing is already extracted; remaining inline blocks (velocity calc + grouping) are tested via the public API.
- **compiler.compile_highlights** (202L): **strong** coverage via `test_compiler.py` + `test_watermark.py` (trim loop, watermark fallback, concat invariant all asserted).
- **person_detector**: well-mocked tests, but grouping logic differs structurally from tracknet ? dedup would be a redesign, not a pure move.

45. user (2026-06-18T07:57:17.531Z)

Structured output provided successfully

46. assistant (2026-06-18T07:57:31.289Z)

I've completed the scoping analysis and returned the structured proposals. Here's the summary of what I found in the cluster:

Strongest safe refactor (high-value, low-risk, well-tested):

1. **Extract the per-segment trim loop in `compiler.compile\_highlights`** ? the longest function in the cluster (202L). The ~100-line loop body extracts cleanly into `_trim_one_segment`. Critically, `tests/test_watermark.py` + `tests/test_compiler.py` assert on the exact ffmpeg argv sequence, the watermark-then-plain retry order, dedup, and the concat `-c copy` invariant ? so a regression would be caught.

Good secondary targets (well-tested pure logic in `tracknet\_runner.py`):

2. **Dedup the temporal-midpoint split-index** copy-pasted between `_split_overlong_groups` and `_blob_fallback_chop` ? both covered by direct pure unit tests.
3. **Extract `\_active\_frames`** (the velocity/visibility loop) out of `trajectory_to_action_windows` (231L) ? exercised end-to-end by the moving/stationary/gap-split trajectory tests.

Explicitly flagged NOT-safe-yet:

4. **wasb_infer.run()** (185L) ? tempting detector-pass-loop extraction, but it has **zero** automated coverage (WSL/GPU-only; `test_wasb_cache.py` covers only the surrounding pure helpers). The flush-cadence/manifest/resume-index bookkeeping is exactly the stateful logic an extraction can silently break. Marked **needs tests first**.
5. **person_detector** ? tracknet grouping dedup ? surface-similar but the tuple shapes and `_finalize` stat computation (`track_id_count` set-union vs hardcoded 1) genuinely differ; unifying is a redesign, not a pure move.

Baseline confirmed green: 82 tests pass across the five relevant test files. The full proposal details (seams, behavior-preservation arguments, honest coverage, risk, diff sizes) are in the StructuredOutput.